

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

-----□□□□-----



BÁO CÁO

CƠ SỞ DỮ LIỆU PHÂN TÁN

Giảng viên hướng dẫn: Kim Ngọc Bách

Sinh viên thực hiện: Bùi Hồng Phú - B23DCCN651

Nguyễn Xuân Lộc - B23DCCN492

Trần Việt Anh - B23DCCN050

Nhóm lớp: 10

Nhóm thực hiện: 12

Hà Nội, 2026

Mục lục

Mục lục.....	2
Các hình ảnh.....	5
Danh mục các từ viết tắt.....	6
I. Phân tích nghiệp vụ.....	11
1.1. Bối cảnh bài toán.....	11
1.2. Mục tiêu của hệ thống.....	11
1.2.1 Mục tiêu nghiệp vụ.....	11
1.2.2 Mục tiêu phân tán.....	12
1.3. Các thực thể nghiệp vụ chính.....	12
1.3.1 Dữ liệu dùng chung (replicated data).....	12
1.3.2 Dữ liệu cục bộ theo site (fragmented data).....	13
1.4. Chức năng nghiệp vụ chính.....	13
1.5. Kiến trúc ba lớp của hệ thống demo.....	14
II. Lược đồ toàn cục.....	14
2.1. Quan điểm thiết kế.....	14
2.2. Danh sách bảng và vai trò.....	15
2.2.1 Nhóm bảng nhân bản ở mọi site.....	15
2.2.2 Nhóm bảng phân mảnh theo site.....	15
2.3. Ràng buộc logic giữa các bảng.....	16
2.4. Sơ đồ lược đồ toàn cục.....	18
III. Phân mảnh và cấp phát dữ liệu.....	18
3.1. Mục đích và nguyên tắc phân mảnh.....	18
3.2. Mô hình ba site.....	19
3.3. Phân mảnh ngang theo site (Horizontal Fragmentation).....	19
3.3.1 Biểu thức phân mảnh hình thức.....	19
3.3.2 Kiểm tra tính đúng đắn của phân mảnh.....	20
3.4. Ý nghĩa của việc phân mảnh.....	20
3.4.1 Tránh dữ liệu trùng lặp.....	20
3.4.2 Giảm chi phí đồng bộ.....	20
3.4.3 Giảm nguy cơ inconsistency.....	21
3.5. Nhân bản dữ liệu dùng chung.....	21
3.6. Cấp phát dữ liệu thực tế trong hệ thống demo.....	21
3.7. Sơ đồ phân mảnh và cấp phát.....	22
3.8. Liên hệ với triển khai thực tế.....	22
VI. Cơ chế nhân bản và đồng bộ dữ liệu.....	22

4.1. Mục tiêu của cơ chế nhân bản.....	22
4.2. Chiến lược nhân bản.....	23
4.3. Dữ liệu được nhân bản và dữ liệu không nhân bản.....	23
4.4. Đồng bộ logic thông qua FastAPI Coordinator.....	24
4.5. Đồng bộ trong truy vấn đọc.....	24
4.6. Đồng bộ trong truy vấn ghi.....	25
4.7. Ưu điểm và hạn chế của cách làm hiện tại.....	26
V. Truy vấn phân tán.....	27
5.1. Mục đích và ý nghĩa.....	27
5.2. Sơ đồ xử lý truy vấn phân tán tổng quát.....	27
5.3. Truy vấn tồn kho toàn hệ thống.....	28
5.4. Truy vấn quote phân bổ đơn hàng.....	29
5.5. Truy vấn doanh thu theo site.....	31
5.6. Truy vấn top sản phẩm bán chạy.....	32
5.7. Truy vấn đơn hàng đa kho.....	33
5.8. Bảng tổng hợp truy vấn phân tán.....	34
VI. Tình huống phân tán tiêu biểu.....	35
6.1. Tra cứu tồn kho toàn hệ thống.....	35
6.2. Đặt hàng khi kho gần nhất không đủ hàng.....	36
Trường hợp lỗi.....	38
VII. Đồng thời và nhất quán tồn kho.....	39
7.1. Bài toán đặt ra.....	39
7.2. Khái niệm nhất quán trong demo.....	39
7.3. Mô hình available_qty và reserved_qty.....	39
7.4. Cơ chế SELECT ... FOR UPDATE.....	40
7.5. Chu trình reserve/commit/release.....	41
7.5.1 Reserve – Giữ chỗ tồn kho.....	41
7.5.2 Commit – Xác nhận giao dịch thành công.....	41
7.5.3 Release – Hoàn trả khi giao dịch thất bại.....	41
7.5.4 Sơ đồ chu trình reserve/commit/release.....	41
7.6. Kịch bản đồng thời trong bản demo.....	42
7.7. Vai trò của bảng inventory_audit.....	43
7.8. Mức độ hoàn chỉnh của giải pháp.....	44
VIII. Cài đặt hệ thống.....	45
8.1. Cài đặt hạ tầng web.....	45
8.1.1. Khởi động cụm database.....	45

8.1.2. Khởi động FastAPI.....	45
8.1.3. Khởi động frontend.....	46
8.1.4. Khởi tạo dữ liệu mẫu.....	46
8.2. Cài đặt logic phân tán.....	47
8.2.1. Phân tán dữ liệu qua seed khác nhau trên từng site.....	47
8.2.2. Coordinator kết nối đồng thời ba site.....	47
8.2.3. Truy vấn phân tán fan-out.....	48
8.2.4. Cơ chế nhất quán tồn kho trong môi trường đồng thời.....	49
8.2.5. Audit log phân tán.....	50
IX. Đánh giá hệ thống.....	51
9.1. Kết quả kiểm thử thực tế.....	51
9.1.1. Kiểm tra health 3 site.....	51
9.1.2. Tra cứu tồn kho toàn hệ thống (fan-out).....	51
9.1.3. Quote phân bổ theo vùng địa lý (locality of reference).....	52
9.1.4. Tạo đơn hàng split order (điều phối đa site).....	52
9.1.5. Xem allocation timeline (audit log).....	53
9.1.6. Demo đồng thời (nhất quán tồn kho).....	53
9.1.7. Báo cáo tổng hợp đa site.....	54
9.2. Kết quả kiểm thử tự động.....	55
9.3. Đánh giá mức độ đáp ứng mục tiêu phân tán.....	55
9.4. Phân tích lý thuyết và hạn chế.....	56
X. Kết luận và hướng phát triển.....	57
10.1. Kết luận.....	57
10.2. Hướng phát triển.....	58
10.2.1. Nâng cấp cơ chế giao dịch phân tán.....	58
10.2.2. Cải thiện độ tin cậy.....	58
10.2.3. Mở rộng quy mô và tính năng.....	58

Các hình ảnh

Hình 1. Sơ đồ toàn cục.....	18
Hình 2. Phân mảnh, nhân bản và cấp phát.....	22
Hình 3. Sơ đồ vai trò của coordinator.....	24
Hình 4. Sơ đồ truy vấn phân tán đọc.....	25
Hình 5. Sơ đồ truy vấn phân tán ghi.....	26
Hình 6. Kiểm tra sản phẩm còn ở những kho nào.....	28
Hình 7. Phân bổ đơn hàng theo vùng khách hàng.....	29
Hình 8. Sơ đồ truy vấn quote phân bổ hàng.....	31
Hình 9. Sơ đồ truy vấn doanh thu theo site.....	32
Hình 10. Sơ đồ truy vấn top sản phẩm bán chạy.....	33
Hình 11. Sơ đồ truy vấn đơn hàng đa kho.....	34
Hình 12. Sơ đồ tra cứu tồn kho toàn hệ thống.....	36
Hình 13. Sơ đồ đặt hàng khi kho không đủ hàng.....	37
Hình 14. Sơ đồ chu trình reserve/commit/release.....	42
Hình 15. Sơ đồ tuần tự concurrency.....	43
Hình 16. Khởi chạy thành công ba container.....	45
Hình 17. Khởi chạy thành công API.....	46
Hình 18. Khởi chạy thành công frontend.....	46
Hình 19. Tạo dữ liệu thành công cho 3 site.....	46
Hình 20. Kiểm tra health.....	51
Hình 21. Kiểm tra số lượng hàng mã LAP-01 cả hệ thống.....	52
Hình 22. Phân bổ thành công.....	52
Hình 23. Tạo đơn hàng LAP-01 cho khách CUS-N-01 thành công.....	53
Hình 24. Kết quả allocation_logs của mã ORD-15D8294F.....	53
Hình 25. Kết quả demo chống âm kho.....	54

Danh mục các từ viết tắt

STT	Thuật ngữ/Từ viết tắt	Viết đầy đủ	Ý nghĩa
1	CSDL	Cơ Sở Dữ Liệu	Tập hợp có tổ chức các dữ liệu được lưu trữ và quản lý để phục vụ truy xuất và xử lý thông tin.
2	CSDLPT	Cơ Sở Dữ Liệu Phân Tán	Hệ thống cơ sở dữ liệu trong đó dữ liệu được lưu trữ trên nhiều địa điểm vật lý khác nhau nhưng được quản lý như một hệ thống thống nhất.
3	SKU	Stock Keeping Unit	Mã định danh duy nhất cho từng loại sản phẩm, dùng để quản lý và theo dõi hàng hóa trong kho.
4	API	Application Programming Interface	Giao diện lập trình ứng dụng cho phép các phần mềm giao tiếp và trao đổi dữ liệu với nhau.
5	FastAPI	Fast Application Programming Interface	Framework Python hiệu năng cao dùng để xây dựng các web API, đóng vai trò coordinator trong hệ thống phân tán của đề tài.

6	REST	Representational State Transfer	Kiến trúc phần mềm định nghĩa các ràng buộc để tạo ra dịch vụ web, sử dụng giao thức HTTP để truyền dữ liệu.
7	HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản, là nền tảng của giao tiếp dữ liệu trên World Wide Web.
8	DSN	Data Source Name	Chuỗi cấu hình chứa thông tin kết nối đến cơ sở dữ liệu, bao gồm host, port, tên database và thông tin xác thực.
9	PostgreSQL	PostgreSQL (Post-gres-Q-L)	Hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, được sử dụng để lưu trữ dữ liệu tại ba site trong hệ thống demo.
10	Docker	Docker Container Platform	Nền tảng ảo hóa cấp ứng dụng cho phép đóng gói và triển khai ứng dụng trong các container độc lập.
11	2PC	Two-Phase Commit	Giao thức cam kết hai pha trong hệ thống phân tán, đảm bảo tính nhất quán khi một giao dịch cần được thực thi trên nhiều node.

12	CDC	Change Data Capture	Kỹ thuật theo dõi và ghi lại các thay đổi dữ liệu trong cơ sở dữ liệu để phục vụ đồng bộ hóa giữa các hệ thống.
13	PK	Primary Key	Khóa chính – thuộc tính hoặc tập thuộc tính định danh duy nhất cho mỗi bản ghi trong một bảng dữ liệu.
14	FK	Foreign Key	Khóa ngoại – thuộc tính trong một bảng tham chiếu đến khóa chính của bảng khác, thiết lập ràng buộc quan hệ.
15	SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc dùng để quản lý và thao tác với cơ sở dữ liệu quan hệ.
16	DBMS	Database Management System	Hệ quản trị cơ sở dữ liệu – phần mềm cho phép tạo, quản lý và truy xuất dữ liệu trong cơ sở dữ liệu.
17	CAP	Consistency, Availability, Partition tolerance	Định lý CAP trong hệ phân tán: một hệ thống không thể đồng thời đảm bảo cả ba tính chất nhất quán, khả dụng và chịu lỗi phân vùng.
18	DSL	Domain-Specific Language	Ngôn ngữ lập trình chuyên biệt cho một lĩnh vực hoặc mục đích cụ thể.

19	ORM	Object-Relational Mapping	Kỹ thuật ánh xạ giữa đối tượng trong lập trình hướng đối tượng và bảng trong cơ sở dữ liệu quan hệ.
20	React	React JavaScript Library	Thư viện JavaScript mã nguồn mở dùng để xây dựng giao diện người dùng, được sử dụng làm frontend trong hệ thống demo.
21	UI	User Interface	Giao diện người dùng – phần của hệ thống mà người dùng tương tác trực tiếp.
22	npm	Node Package Manager	Trình quản lý gói phần mềm cho môi trường JavaScript/Node.js, dùng để cài đặt và quản lý các thư viện.
23	pip	Pip Installs Packages	Trình quản lý gói phần mềm cho ngôn ngữ lập trình Python.
24	ENV	Environment Variable	Biến môi trường – cặp tên/giá trị được thiết lập trong môi trường hệ điều hành, dùng để cấu hình ứng dụng mà không cần mã hóa cứng.
25	DXA	Document eXtended Attribute (đơn vị twip)	Đơn vị đo lường trong tài liệu Word, với 1440 DXA tương đương 1 inch; dùng để xác định kích thước trang và

			bảng.
26	SUM	Summation Function (SQL)	Hàm tổng hợp trong SQL, tính tổng giá trị của một cột trong tập kết quả truy vấn.
27	GROUP BY	Group By Clause (SQL)	Mệnh đề trong SQL dùng để nhóm các hàng có cùng giá trị ở một hoặc nhiều cột, thường dùng cùng hàm tổng hợp.
28	JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu nhẹ, dễ đọc với con người và dễ phân tích với máy tính, phổ biến trong API.
29	GET	HTTP GET Method	Phương thức HTTP dùng để yêu cầu đọc dữ liệu từ một tài nguyên trên server mà không thay đổi trạng thái.
30	POST	HTTP POST Method	Phương thức HTTP dùng để gửi dữ liệu lên server nhằm tạo mới hoặc xử lý tài nguyên.

I. Phân tích nghiệp vụ

1.1. Bối cảnh bài toán

Doanh nghiệp thương mại điện tử trong đề tài có nhiều kho hàng đặt tại các khu vực địa lý khác nhau. Mỗi kho vận hành tương đối độc lập trong việc quản lý tồn kho, xử lý đơn hàng và cập nhật trạng thái hàng hóa. Tuy nhiên, phía người dùng cuối và quản trị viên vẫn cần một góc nhìn toàn hệ thống để thực hiện các nghiệp vụ xuyên kho và xuyên vùng.

Cụ thể, các nhu cầu đặt ra đối với hệ thống bao gồm:

- + Tra cứu tồn kho của một sản phẩm trên toàn quốc, không giới hạn theo từng kho riêng lẻ.
- + Đặt hàng theo vùng khách hàng, ưu tiên giao hàng từ kho gần nhất với khách.
- + Xử lý trường hợp kho gần nhất không đủ hàng bằng cách lấy bổ sung từ các kho khác.
- + Tổng hợp báo cáo doanh thu từ nhiều site thành một góc nhìn thống nhất cho quản trị viên.
- + Đảm bảo hệ thống vẫn nhất quán khi có nhiều giao dịch đồng thời trên cùng một sản phẩm.

Trong hệ thống demo hiện tại, ba site được mô phỏng tương ứng với ba miền địa lý:

- + north – đại diện cho khu vực miền Bắc
- + central – đại diện cho khu vực miền Trung
- + south – đại diện cho khu vực miền Nam

1.2. Mục tiêu của hệ thống

Đề tài không chỉ yêu cầu tạo một cơ sở dữ liệu lưu trữ thông tin bán hàng, mà còn yêu cầu thể hiện rõ đặc trưng của hệ quản trị cơ sở dữ liệu phân tán. Vì vậy, mục tiêu của hệ thống bao gồm hai lớp chính.

1.2.1 Mục tiêu nghiệp vụ

- Quản lý sản phẩm và danh mục sản phẩm: cho phép xem danh sách sản phẩm, xem chi tiết theo SKU và phân loại theo danh mục.
- Quản lý thông tin kho hàng theo vùng: theo dõi kho nào thuộc site nào, thuộc thành phố và vùng nào.

- Quản lý tồn kho tại từng kho: theo dõi số lượng tồn, số lượng đang giữ chỗ và ngưỡng reorder.
- Quản lý khách hàng: lưu trữ thông tin khách hàng gắn với vùng địa lý để phục vụ logic phân bổ đơn hàng.
- Tạo và theo dõi đơn hàng: hỗ trợ tạo đơn kể cả khi cần lấy hàng từ nhiều kho khác nhau và truy xuất lại chi tiết đơn sau khi tạo.
- Hỗ trợ báo cáo quản trị: doanh thu theo site, top sản phẩm bán chạy, đơn hàng được phân bổ từ nhiều kho.

1.2.2 Mục tiêu phân tán

- Phân bổ dữ liệu theo site: mỗi site lưu trữ phần dữ liệu mà nó sở hữu, không tập trung toàn bộ vào một điểm.
- Nhân bản dữ liệu dùng chung: các bảng catalog ít thay đổi được sao chép tới mọi site để tránh truy vấn xuyên site mỗi lần hiển thị.
- Hỗ trợ truy vấn toàn hệ thống thông qua middleware: người dùng không cần biết dữ liệu nằm ở site nào, middleware sẽ tổng hợp và trả về kết quả thống nhất.
- Giảm rủi ro âm tồn kho khi có giao dịch đồng thời: áp dụng cơ chế khóa dòng và reserve/commit/release để đảm bảo nhất quán.
- Mô phỏng điều phối đơn hàng giữa nhiều site: khi kho ưu tiên không đủ hàng, hệ thống tự động gom từ nhiều site để đáp ứng đơn hàng.

1.3. Các thực thể nghiệp vụ chính

1.3.1 Dữ liệu dùng chung (replicated data)

Nhóm dữ liệu này ít thay đổi nhưng được đọc rất thường xuyên ở mọi site, do đó được nhân bản toàn phần:

- + categories: lưu mã danh mục, tên danh mục và mô tả. Dùng để phân nhóm sản phẩm trên giao diện và trong báo cáo.
- + products: lưu SKU, tên sản phẩm, giá và danh mục. Là bảng trung tâm của catalog – mọi nghiệp vụ từ tồn kho đến đặt hàng và báo cáo đều phụ thuộc vào SKU.
- + warehouses: lưu mã kho, site sở hữu, tên kho, thành phố và vùng. Vừa là dữ liệu dùng chung vừa là cầu nối giữa tầng logic toàn cục và tầng lưu trữ cục bộ.

1.3.2 Dữ liệu cục bộ theo site (fragmented data)

Nhóm dữ liệu này mang tính giao dịch, cập nhật thường xuyên và gắn chặt với từng site:

- + customers: khách hàng được phân theo vùng địa lý, lưu tại site tương ứng với vùng của khách.

- + inventory: tồn kho theo từng kho và SKU, lưu số lượng khả dụng (available_qty) và số lượng đang giữ chỗ (reserved_qty).

- + orders: thông tin đơn hàng gắn với site xử lý chính (primary_site_code), dù đơn hàng có thể được cấp phát từ nhiều site.

- + order_items: chi tiết từng dòng sản phẩm trong đơn hàng, bao gồm SKU, số lượng, đơn giá và thành tiền.

- + allocation_logs: log phân bổ hàng, ghi nhận kho nào đã cấp bao nhiêu đơn vị cho từng đơn hàng. Là bằng chứng của split order đa kho.

- + inventory_audit: log reserve, commit, release tồn kho. Lưu trạng thái trước và sau mỗi thao tác, phục vụ kiểm tra tính nhất quán.

1.4. Chức năng nghiệp vụ chính

- Quản lý danh mục và sản phẩm

- + Xem danh sách sản phẩm trong toàn hệ thống.

- + Xem chi tiết sản phẩm theo mã SKU.

- + Phân loại sản phẩm theo danh mục để hỗ trợ tìm kiếm và thống kê.

- Quản lý kho và tồn kho

- + Xem số lượng tồn tại từng kho cụ thể.

- + Xem tổng tồn trên toàn hệ thống cho một SKU (tổng hợp từ nhiều site).

- + Phát hiện bản ghi tồn kho thấp hơn hoặc bằng ngưỡng reorder để cảnh báo bổ sung hàng.

- Quản lý khách hàng và đơn hàng

- + Tạo đơn hàng theo mã khách hàng với SKU và số lượng cụ thể.

- + Xác định site ưu tiên dựa theo vùng của khách hàng để ưu tiên giao hàng gần nhất.

- + Tạo đơn hàng ngay cả khi cần lấy hàng từ nhiều site khác nhau (split order).
- + Truy xuất lại chi tiết đơn hàng và allocation sau khi tạo thành công.
- Báo cáo quản trị
 - + Doanh thu theo từng site và tổng toàn hệ thống.
 - + Top sản phẩm bán chạy nhất trên toàn hệ thống.
 - + Danh sách đơn hàng được phân bổ từ nhiều kho (multi-warehouse orders).
- Mô phỏng đồng thời
 - + Hai khách hàng cùng đặt cùng một SKU trong cùng thời điểm.
 - + Hệ thống phải đảm bảo không âm tồn kho dù có race condition.
 - + Request thất bại phải được rollback đúng cách, không để lại trạng thái treo.

1.5. Kiến trúc ba lớp của hệ thống demo

Bản demo thể hiện được ba lớp hoàn chỉnh của một hệ thống cơ sở dữ liệu phân tán:

- Lớp dữ liệu: có lược đồ toàn cục rõ ràng, dữ liệu được nhân bản (categories, products, warehouses) và dữ liệu được phân mảnh (customers, inventory, orders, ...) theo từng site.
- Lớp điều phối: FastAPI đóng vai trò coordinator, gom và phân phối dữ liệu giữa các site, thực hiện logic reserve/commit/release và đảm bảo nhất quán giao dịch.
- Lớp trình diễn: frontend React hiển thị rõ các luồng phân tán một cách trực quan, bao gồm tra cứu tồn kho toàn cầu, tạo split order và xem báo cáo đa site.

II. Lược đồ toàn cục

2.1. Quan điểm thiết kế

Lược đồ toàn cục là góc nhìn logic thống nhất của toàn bộ hệ thống, không phụ thuộc vào việc dữ liệu đang được lưu ở site nào. Người thiết kế trước hết phải mô tả toàn bộ dữ liệu như một hệ thống thống nhất, sau đó mới quyết định bảng nào cần nhân bản và bảng nào cần phân mảnh.

Lược đồ toàn cục được xây dựng theo ba nguyên tắc chính:

- + Dữ liệu danh mục là dùng chung trên toàn hệ thống: mọi site đều cần hiểu cùng một sản phẩm, cùng một kho.

- + Dữ liệu tồn kho và giao dịch là cục bộ theo site: mỗi site chỉ chịu trách nhiệm với dữ liệu của mình.

- + Middleware đóng vai trò nối các phần cục bộ lại thành hành vi toàn cục: người dùng nhìn thấy một hệ thống thống nhất dù dữ liệu phân tán.

2.2. Danh sách bảng và vai trò

2.2.1 Nhóm bảng nhân bản ở mọi site

Đây là nhóm bảng ít thay đổi nhưng được truy xuất rất thường xuyên ở mọi site:

Bảng	Các trường chính	Vai trò
categories	category_code (PK), name, description	Phân nhóm sản phẩm. Dữ liệu tham chiếu tĩnh, xuất hiện giống nhau ở mọi site.
products	sku (PK), category_code (FK), name, price, is_active	Catalog sản phẩm toàn hệ thống. Mọi nghiệp vụ tồn kho, đặt hàng, báo cáo đều phụ thuộc vào SKU.
warehouses	warehouse_code (PK), site_code, warehouse_name, city, region_code, is_active	Mô tả kho nào thuộc site nào. Cầu nối giữa tầng logic toàn cục và lưu trữ cục bộ.

2.2.2 Nhóm bảng phân mảnh theo site

Đây là nhóm bảng giao dịch, cập nhật liên tục và gắn với từng site cụ thể:

Bảng	Các trường chính	Vai trò
customers	customer_code (PK), full_name, phone, email, city, region_code	Lưu khách hàng theo vùng. Phục vụ xác định site ưu tiên khi đặt hàng.

inventory	warehouse_code (PK), sku (PK), available_qty, reserved_qty, reorder_level, updated_at	Tồn kho tại từng kho. Khóa chính kép đảm bảo mỗi kho chỉ có một bản ghi cho mỗi SKU.
orders	order_code (PK), customer_code (FK), primary_site_code, primary_warehouse_code , status, total_amount, created_at	Thông tin đầu đơn hàng. Gắn với một primary_site nhưng có thể cấp phát từ nhiều site.
order_items	order_code (PK), sku (PK), quantity, unit_price, line_total	Chi tiết dòng hàng trong đơn. Khóa chính kép đảm bảo mỗi SKU chỉ xuất hiện một lần trong đơn.
allocation_logs	id (PK), order_code (FK), sku (FK), site_code, warehouse_code (FK), allocated_qty, action, created_at	Log phân bổ hàng từ nhiều kho. Bằng chứng của split order đa kho.
inventory_audit	id (PK), request_code, sku, warehouse_code, action, delta_available, delta_reserved, created_at	Dấu vết thay đổi tồn kho. Ghi lại từng thao tác reserve, commit, release.

2.3. Ràng buộc logic giữa các bảng

Quan hệ categories – products (1 - N): Một danh mục có thể chứa nhiều sản phẩm, nhưng một sản phẩm chỉ thuộc đúng một danh mục. Quan hệ này cho phép phân loại sản phẩm theo nhóm để hỗ trợ tìm kiếm, thống kê và hiển thị trên giao diện người dùng.

Quan hệ warehouses – inventory (1 - N): Một kho có thể lưu nhiều SKU khác nhau, nhưng mỗi bản ghi inventory thuộc đúng một kho. Quan hệ này cho phép theo dõi tồn kho theo từng địa điểm vật lý cụ thể.

Quan hệ products – inventory (1 - N): Một SKU có thể tồn tại ở nhiều kho khác nhau (ở cả 3 site), nhưng mỗi bản ghi inventory chỉ quản lý một SKU. Đây là nền tảng của bài toán đa kho: hệ thống có thể truy vấn tổng tồn toàn hệ thống, thực hiện split order và điều phối fulfillment giữa nhiều site.

Quan hệ customers – orders (1 - N): Một khách hàng có thể tạo nhiều đơn hàng, nhưng mỗi đơn hàng thuộc về đúng một khách hàng. Quan hệ này cho phép tra lịch sử mua hàng, thống kê khách hàng và xác định vùng ưu tiên fulfillment theo vùng của khách.

Quan hệ orders – order_items (1 - N): Một đơn hàng có thể chứa nhiều dòng sản phẩm, nhưng mỗi dòng hàng thuộc về đúng một đơn hàng. Quan hệ này tách phần header của đơn hàng (orders) với phần chi tiết từng sản phẩm (order_items) để dễ mở rộng và quản lý.

Quan hệ products – order_items (1 - N): Một sản phẩm có thể xuất hiện trong nhiều đơn hàng khác nhau, nhưng mỗi dòng order_item chỉ tham chiếu đến một SKU. Quan hệ này cho phép thống kê top sản phẩm, tính doanh thu theo SKU và phân tích bán hàng theo danh mục.

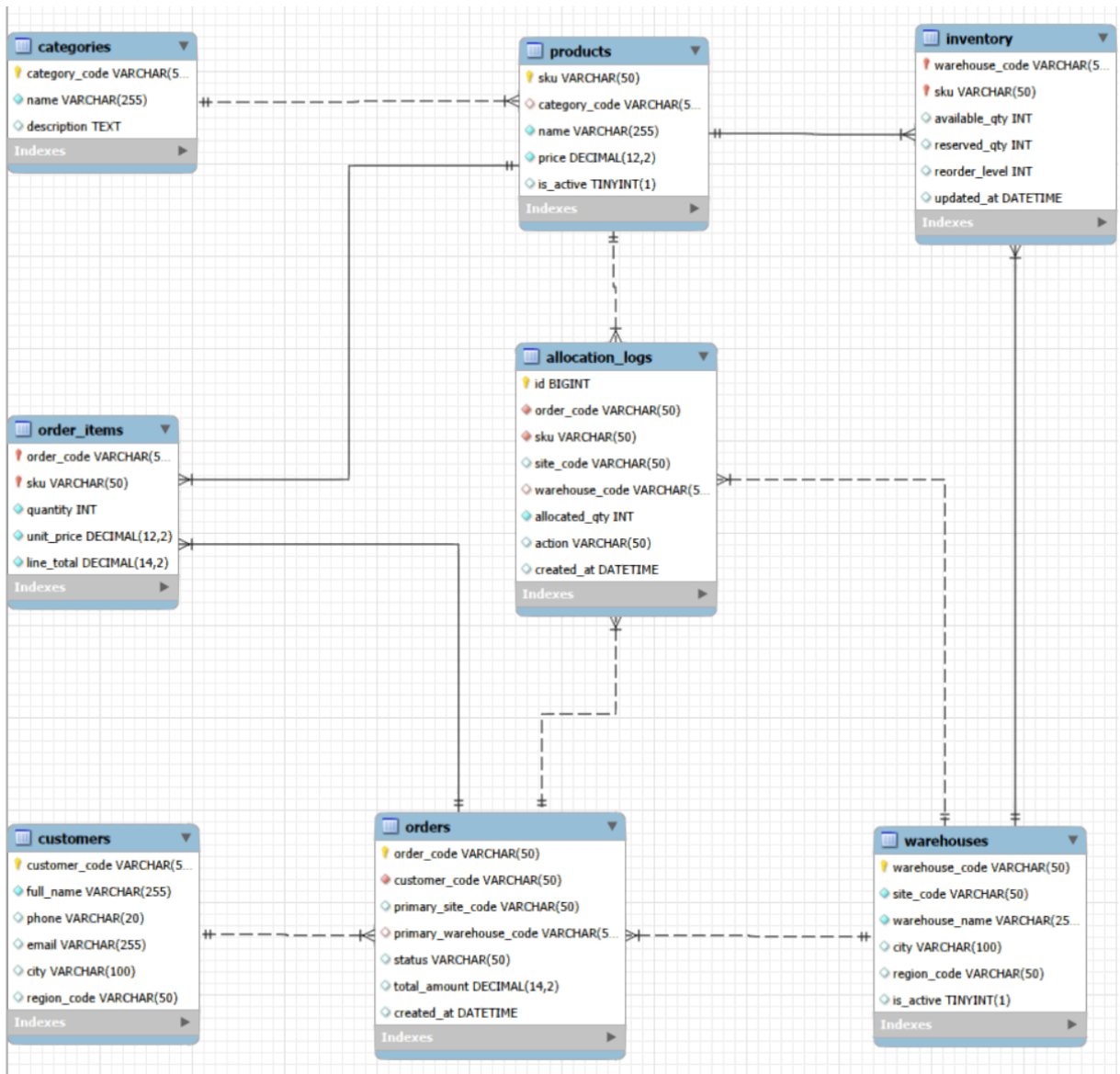
Quan hệ orders – allocation_logs (1 - N): Một đơn hàng có thể được cấp phát bởi nhiều kho khác nhau, mỗi allocation log thuộc về một order cụ thể. Quan hệ này là nền tảng để chứng minh cơ chế multi-warehouse fulfillment, cho phép xác định đơn nào bị split, warehouse nào cấp bao nhiêu và timeline fulfillment của đơn hàng.

Quan hệ products và warehouses – allocation_logs (1 - N): Mỗi allocation log gắn với một SKU cụ thể và một warehouse cụ thể. Điều này cho phép audit cấp phát tồn kho, thống kê lượng fulfillment theo sản phẩm và debug transaction phân tán khi có sự cố.

Khóa chính kép trong inventory và order_items: Bảng inventory sử dụng khóa chính kép (warehouse_code, sku), đảm bảo một kho chỉ có tối đa một bản ghi tồn kho cho mỗi SKU – tránh tình trạng dữ liệu tồn kho bị nhân bản không kiểm soát trong cùng một kho.

Bảng order_items sử dụng khóa chính kép (order_code, sku), đảm bảo trong cùng một đơn hàng mỗi SKU chỉ xuất hiện đúng một lần. Thay vì ghi hai dòng LAP-01 x2 và LAP-01 x3, hệ thống sẽ ghi một dòng duy nhất LAP-01 x5.

2.4. Sơ đồ lược đồ toàn cục



Hình 1. Sơ đồ toàn cục

III. Phân mảnh và cấp phát dữ liệu

3.1. Mục đích và nguyên tắc phân mảnh

Trong hệ phân tán, không phải mọi dữ liệu đều nên đặt ở mọi nơi. Nếu toàn bộ dữ liệu được nhân bản toàn phần, hệ thống sẽ tốn chi phí đồng bộ rất lớn và dễ phát sinh xung đột khi nhiều site cùng cập nhật một dữ liệu. Nếu ngược lại mọi dữ liệu đều chỉ tồn tại ở một chỗ, truy vấn xuyên site sẽ xảy ra thường xuyên, làm tăng độ trễ và giảm hiệu năng. Do đó cần lựa chọn mô hình cân bằng: nhân bản dữ liệu dùng chung và phân mảnh dữ liệu giao dịch theo site.

3.2. Mô hình ba site

Site	Khu vực	Vai trò
north	Miền Bắc	Xử lý kho và khách hàng miền Bắc
central	Miền Trung	Xử lý kho và khách hàng miền Trung
south	Miền Nam	Xử lý kho và khách hàng miền Nam

3.3. Phân mảnh ngang theo site (Horizontal Fragmentation)

Áp dụng cho các bảng: customers, inventory, orders, order_items, allocation_logs và inventory_audit. Mỗi site giữ phần dữ liệu mà nó sở hữu – khách hàng của vùng đó, tồn kho của kho thuộc site đó, đơn hàng mà site đó xử lý chính và các log nghiệp vụ liên quan.

3.3.1 Biểu thức phân mảnh hình thức

- Khách hàng được phân theo vùng địa lý:
 - + customers_north = $\sigma(\text{region_code} = \text{'north'})$ (customers)
 - + customers_central = $\sigma(\text{region_code} = \text{'central'})$ (customers)
 - + customers_south = $\sigma(\text{region_code} = \text{'south'})$ (customers)
- Tồn kho được phân theo warehouse thuộc từng site:
 - + inventory_north = $\sigma(\text{site_code} = \text{'north'})$ (inventory)
 - + inventory_central = $\sigma(\text{site_code} = \text{'central'})$ (inventory)
 - + inventory_south = $\sigma(\text{site_code} = \text{'south'})$ (inventory)
- Đơn hàng được phân theo site xử lý chính của đơn:
 - + orders_north = $\sigma(\text{primary_site_code} = \text{'north'})$ (orders)
 - + orders_central = $\sigma(\text{primary_site_code} = \text{'central'})$ (orders)
 - + orders_south = $\sigma(\text{primary_site_code} = \text{'south'})$ (orders)
- Log cấp phát (allocation_logs) được phân theo site phát sinh giao dịch:
 - + allocation_logs_north = $\sigma(\text{site_code} = \text{'north'})$ (allocation_logs)
 - + allocation_logs_central = $\sigma(\text{site_code} = \text{'central'})$ (allocation_logs)
 - + allocation_logs_south = $\sigma(\text{site_code} = \text{'south'})$ (allocation_logs)
- Log kiểm tra tồn kho (inventory_audit) cũng được phân theo site nơi

thao tác diễn ra:

+ inventory_audit_north = $\sigma(\text{site_code} = \text{'north'})$ (inventory_audit)
+ inventory_audit_central = $\sigma(\text{site_code} = \text{'central'})$ (inventory_audit)
+ inventory_audit_south = $\sigma(\text{site_code} = \text{'south'})$ (inventory_audit)

3.3.2 Kiểm tra tính đúng đắn của phân mảnh

Theo lý thuyết cơ sở dữ liệu phân tán, chiến lược phân mảnh cần thỏa mãn ba tính chất:

- **Tính đầy đủ (Completeness):** Mọi dữ liệu của bảng gốc phải xuất hiện ở ít nhất một mảnh. Không có dữ liệu nào bị mất sau khi phân mảnh. Ví dụ: customers = customers_north $\cup$ customers_central $\cup$ customers_south.

- **Khả năng tái tạo (Reconstruction):** Có thể tái tạo lại bảng gốc bằng cách hợp nhất các mảnh. Trong hệ thống demo, FastAPI coordinator thực hiện việc tổng hợp này mỗi khi cần truy vấn toàn hệ thống, ví dụ: orders = orders_north $\cup$ orders_central $\cup$ orders_south.

- **Tính không giao nhau (Disjointness):** Một bản ghi chỉ thuộc đúng một mảnh dữ liệu. Mỗi đơn hàng chỉ có một primary_site_code duy nhất nên không thể tồn tại đồng thời ở nhiều site. Điều này giúp tránh dữ liệu trùng lặp và giảm nguy cơ xung đột cập nhật.

3.4. Ý nghĩa của việc phân mảnh

3.4.1 Tránh dữ liệu trùng lặp

Khi dữ liệu giao dịch được phân mảnh theo site, mỗi site chỉ lưu phần dữ liệu mà nó thực sự sở hữu. Ví dụ, customers_north chỉ chứa khách hàng miền Bắc, customers_south chỉ chứa khách hàng miền Nam. Hệ thống không lưu toàn bộ khách hàng của cả nước ở mọi site. Điều này giúp tránh lưu nhiều bản sao không cần thiết, giảm dung lượng database và giảm khả năng dữ liệu bị lệch giữa các site. Ví dụ khi khách hàng CUS-N-01 ở miền Bắc cập nhật số điện thoại, chỉ cần cập nhật tại site north thay vì phải đồng bộ sang tất cả các site.

3.4.2 Giảm chi phí đồng bộ

Nếu tất cả các bảng đều được nhân bản toàn phần giữa các site, mọi thay đổi sẽ cần đồng bộ liên tục. Mỗi lần tạo order kèm order_items, inventory và allocation_logs đều phải replicate sang cả 3 site, làm tăng số lượng network message, số lượng distributed transaction và độ trễ hệ thống. Trong dự án này, các bảng giao dịch được giữ local tại site xử lý nên north chỉ cập nhật dữ liệu

north, south chỉ cập nhật dữ liệu south. Chỉ những bảng ít thay đổi như products, categories và warehouses mới được chia sẻ trên tất cả site, giảm đáng kể chi phí đồng bộ tổng thể.

3.4.3 Giảm nguy cơ inconsistency

Inconsistency xảy ra khi nhiều site giữ nhiều phiên bản khác nhau của cùng một dữ liệu. Ví dụ nếu bảng inventory được sao chép ở cả 3 site nhưng một site chưa kịp đồng bộ, hệ thống có thể đọc sai số lượng thật, dẫn tới overselling, âm tồn kho hoặc báo cáo sai. Vì vậy inventory, orders, order_items và allocation_logs không được replicate toàn phần mà chỉ tồn tại tại site sở hữu. Người dùng vẫn nhìn thấy dữ liệu toàn cục thông qua FastAPI coordinator là lớp tổng hợp kết quả từ nhiều site, giúp dữ liệu local luôn là nguồn sự thật (source of truth), hạn chế xung đột cập nhật và giảm rủi ro dữ liệu không nhất quán.

3.5. Nhân bản dữ liệu dùng chung

Các bảng categories, products và warehouses được nhân bản ở tất cả các site vì chúng thỏa mãn ba tiêu chí: được đọc rất thường xuyên bởi mọi site, thay đổi rất ít so với dữ liệu giao dịch, và cần xuất hiện ở mọi site để tránh join xuyên site mỗi lần tra cứu.

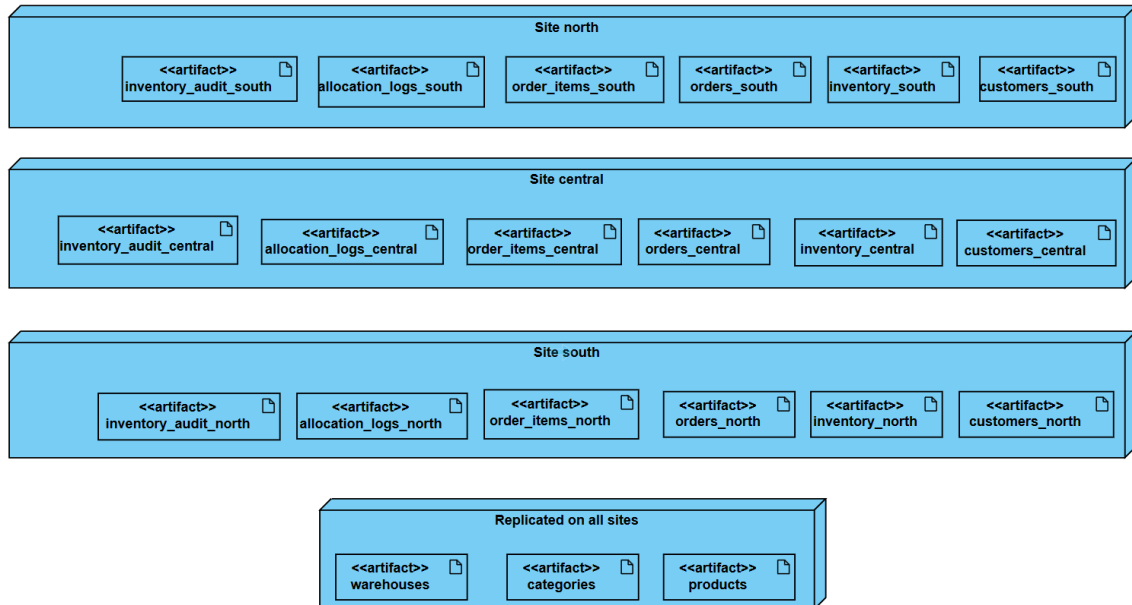
Nhờ nhân bản ba bảng này, frontend và middleware có thể hiển thị tên sản phẩm, tên kho và thông tin site mà không cần thực hiện join xuyên site, giúp giảm độ phức tạp truy vấn và tăng tốc phản hồi.

3.6. Cấp phát dữ liệu thực tế trong hệ thống demo

Site	Dữ liệu cục bộ
north	Khách hàng miền Bắc, tồn kho kho Hà Nội, đơn hàng xử lý chính tại north, log allocation và audit phát sinh tại north.
central	Khách hàng miền Trung, tồn kho kho Đà Nẵng, đơn hàng xử lý chính tại central, log allocation và audit phát sinh tại central.
south	Khách hàng miền Nam, tồn kho kho TP.HCM, đơn hàng xử lý chính tại south, log allocation và audit phát sinh tại south.

Ngoài dữ liệu cục bộ, cả 3 site đều có bản sao đầy đủ của categories, products và warehouses (dữ liệu dùng chung, seed giống nhau).

3.7. Sơ đồ phân mảnh và cấp phát



Hình 2. Phân mảnh, nhân bản và cấp phát

3.8. Liên hệ với triển khai thực tế

Việc phân mảnh dữ liệu trong bản demo không được triển khai bằng distributed DBMS tự động mà được mô phỏng bằng 3 database PostgreSQL độc lập. Mỗi database có schema giống nhau, dữ liệu cục bộ được seed khác nhau theo site, trong khi dữ liệu dùng chung (products, categories, warehouses) được seed giống nhau ở cả 3 site. FastAPI coordinator thực hiện truy vấn và điều phối giữa các site thay cho một distributed DBMS thực thụ.

VI. Cơ chế nhân bản và đồng bộ dữ liệu

4.1. Mục tiêu của cơ chế nhân bản

Nhân bản dữ liệu trong hệ phân tán nhằm giải quyết ba bài toán chính: tăng tốc độ đọc dữ liệu dùng chung bằng cách lưu bản sao tại chỗ, giảm số lần truy vấn xuyên site vốn tốn kém về độ trễ mạng, và giữ cho các site có cùng hiểu biết về catalog sản phẩm và kho.

Tuy nhiên, không phải bảng nào cũng nên nhân bản. Nếu nhân bản cả các bảng giao dịch như orders hay inventory, hệ thống sẽ tốn nhiều công sức đồng bộ và dễ phát sinh xung đột khi nhiều site đồng thời cập nhật cùng một dữ liệu.

4.2. Chiến lược nhân bản

Trong hệ thống thực tế, replication thường được triển khai theo hai hướng:

- **Synchronous replication (đồng bộ tức thời):** Khi dữ liệu được ghi tại một site, transaction chỉ được xem là hoàn tất nếu các bản sao ở site khác cũng ghi thành công. Đảm bảo tính nhất quán tuyệt đối nhưng tăng độ trễ ghi.

- **Asynchronous replication (đồng bộ không tức thời):** Site chính ghi dữ liệu trước, việc cập nhật sang các site khác được thực hiện ở thời điểm sau. Độ trễ ghi thấp nhưng trong khoảng thời gian chờ đồng bộ, dữ liệu giữa các site có thể không nhất quán.

Trong phạm vi dự án, hệ thống sử dụng phương pháp logical asynchronous replication được mô phỏng:

- Dữ liệu dùng chung (products, categories, warehouses) được seed giống nhau ở cả 3 site ngay từ đầu.

- Coordinator giả định catalog dữ liệu luôn nhất quán giữa các site.

- Không triển khai replication engine thời gian thực giữa các node PostgreSQL.

Lý do lựa chọn cách tiếp cận này: phù hợp phạm vi dự án, tập trung minh họa logic phân tán, tránh phụ thuộc vào cấu hình replication của một DBMS cụ thể. Hạn chế là thay đổi catalog chưa tự động lan sang site khác và cần reseed hoặc cập nhật thủ công nếu dữ liệu shared thay đổi.

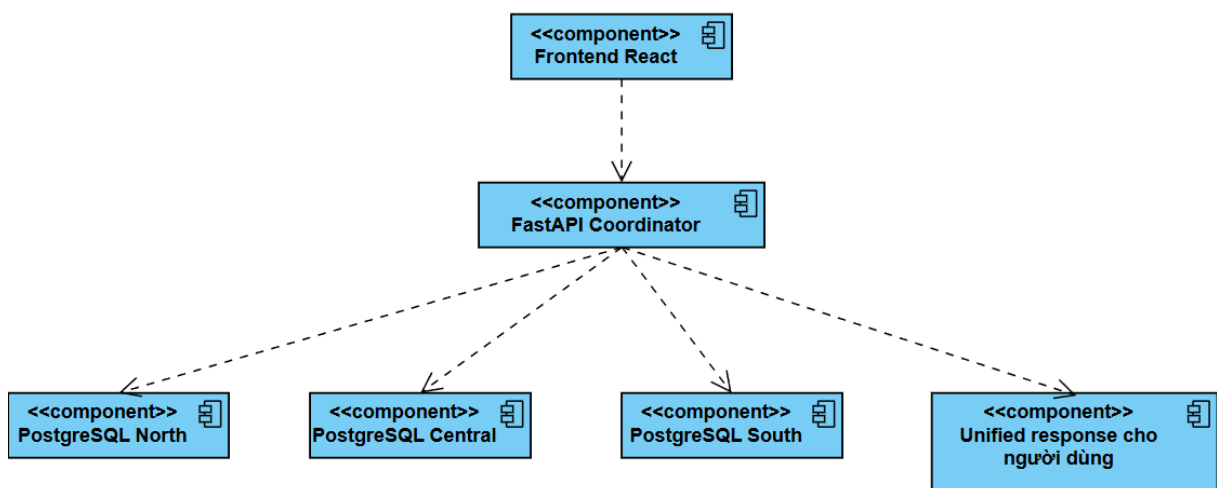
4.3. Dữ liệu được nhân bản và dữ liệu không nhân bản

Loại	Bảng	Lý do
Nhân bản toàn phần	categories, products, warehouses	Ít thay đổi, đọc thường xuyên, cần có mặt ở mọi site để tránh join xuyên site.
Không nhân bản	customers, inventory, orders, order_items, allocation_logs, inventory_audit	Cập nhật thường xuyên, mang tính cục bộ cao, dễ xung đột nếu sao chép toàn phần.

4.4. Đồng bộ logic thông qua FastAPI Coordinator

FastAPI middleware đóng vai trò coordinator ở giữa frontend và 3 site PostgreSQL. Coordinator không chỉ đơn thuần chuyển tiếp request mà thực hiện nhiều nhiệm vụ phức tạp:

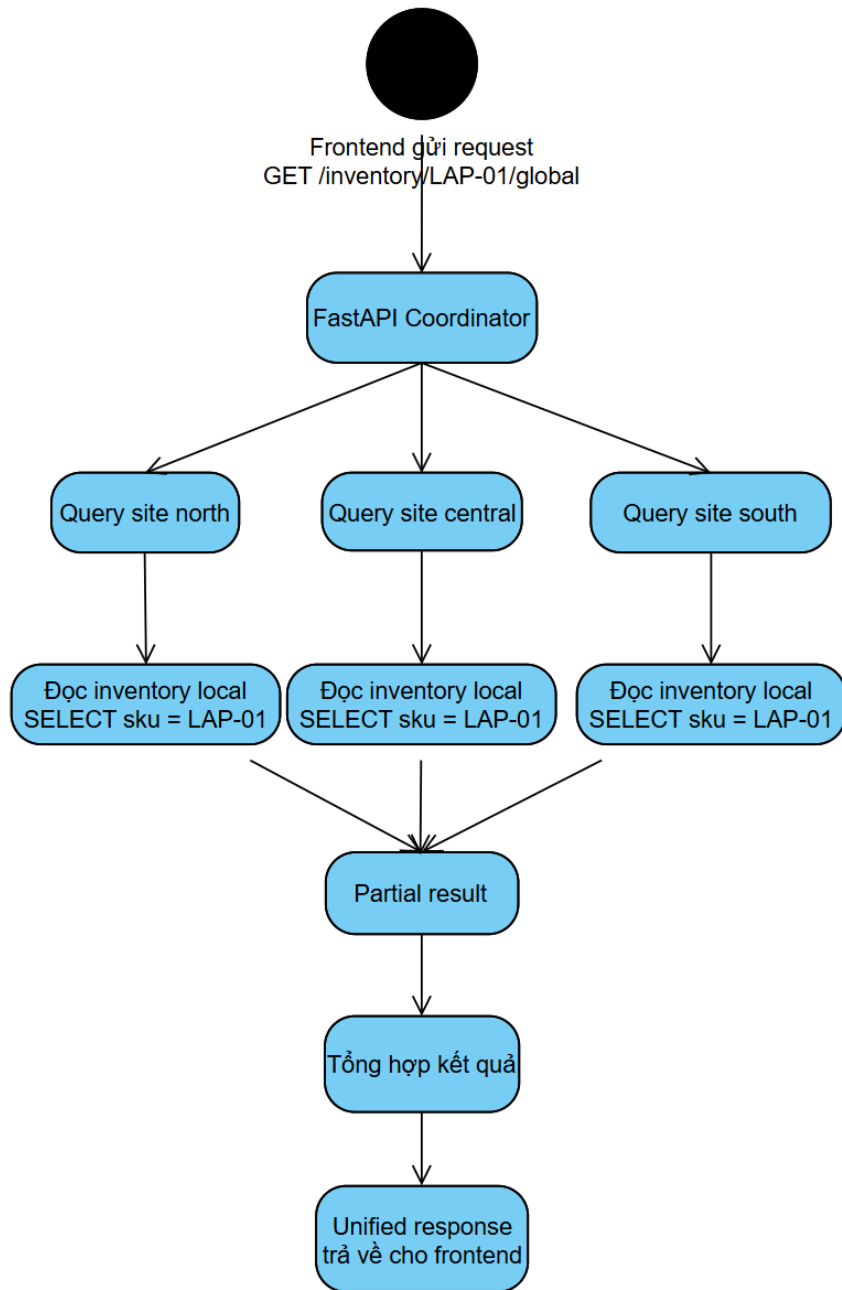
- Gửi truy vấn song song tới nhiều site và gom kết quả về một response chung.
- Tính toán allocation khi một site không đủ hàng, xác định site nào cấp bao nhiêu.
- Giữ chỗ tồn kho bằng reserve trước khi commit, sử dụng SELECT ... FOR UPDATE để tránh race condition.
- Release phần đã reserve và rollback nếu transaction không hoàn tất ở bất kỳ bước nào.



Hình 3. Sơ đồ vai trò của coordinator

4.5. Đồng bộ trong truy vấn đọc

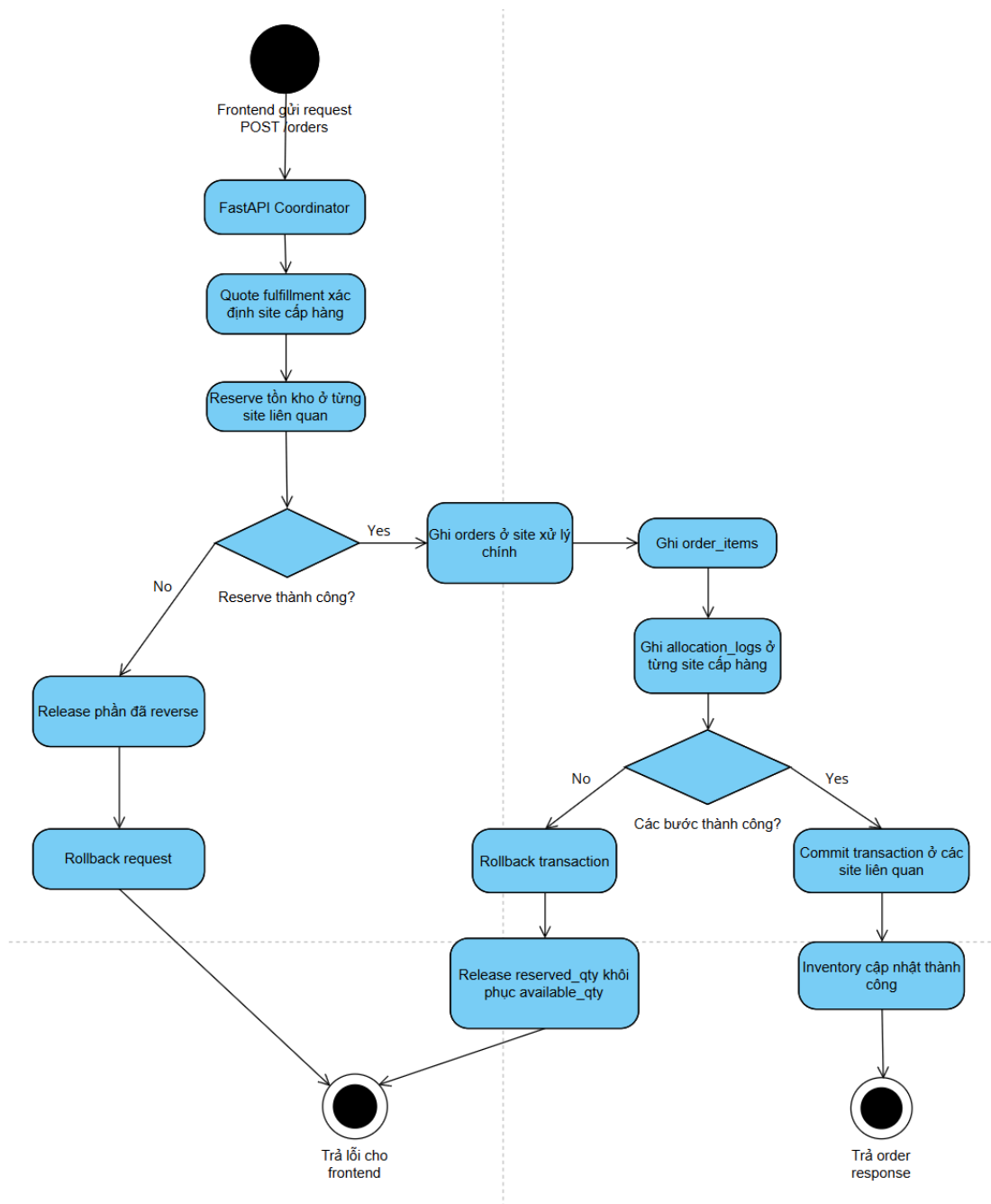
Khi tra cứu tồn kho toàn hệ thống, dữ liệu được đọc cục bộ ở mỗi site, coordinator tổng hợp kết quả mà không cần sao chép bảng inventory sang site khác. Điều này giúp dữ liệu inventory luôn là nguồn sự thật tại chỗ, không có nguy cơ đọc bản sao lỗi thời.



Hình 4. Sơ đồ truy vấn phân tán đọc

4.6. Đồng bộ trong truy vấn ghi

Khi tạo đơn hàng từ nhiều kho, coordinator phải điều phối ghi dữ liệu trên nhiều site theo thứ tự: reserve tồn kho ở từng site liên quan, ghi order ở site xử lý chính, ghi allocation logs, rồi mới commit. Nếu bất kỳ bước nào thất bại, toàn bộ quá trình phải được rollback và dữ liệu trở về trạng thái ban đầu.



Hình 5. Sơ đồ truy vấn phân tán ghi

4.7. Ưu điểm và hạn chế của cách làm hiện tại

- Ưu điểm của phương pháp mô phỏng này:

- + Dễ cài đặt và triển khai trên máy cá nhân mà không cần cấu hình phức tạp.
- + Dễ giải thích sự phân biệt giữa local data và shared data cho người xem.
- + Dễ kiểm soát luồng đọc/ghi giữa các site thông qua coordinator.

- Hạn chế cần nhận thức:

- + Chưa có replication engine thật sự theo thời gian thực giữa các site.

- + Chưa có cơ chế tự đồng bộ khi dữ liệu catalog thay đổi.

- + Chưa triển khai distributed transaction manager hoàn chỉnh kiểu 2PC (Two-Phase Commit) thực thụ.

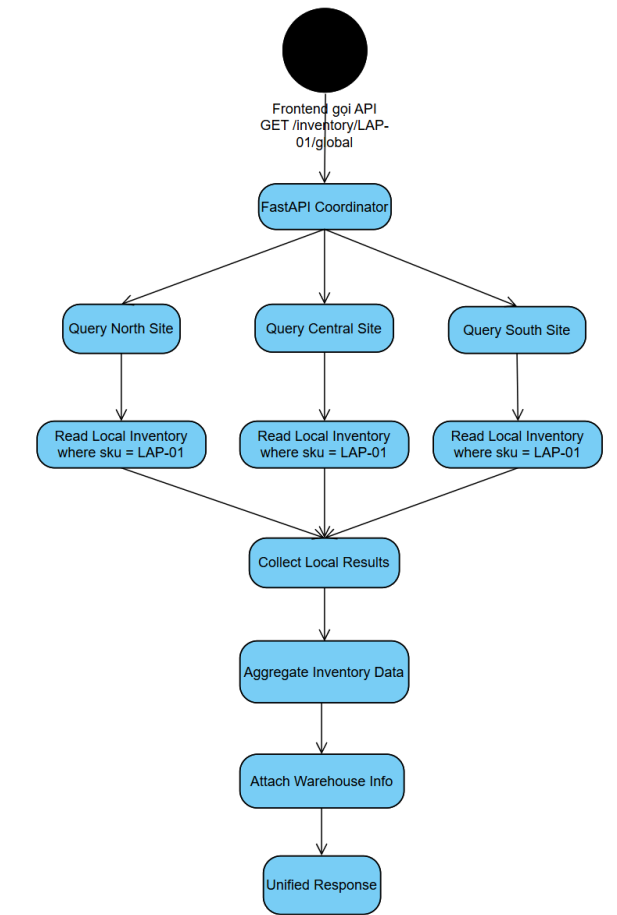
V. Truy vấn phân tán

5.1. Mục đích và ý nghĩa

Một hệ phân tán chỉ thực sự có ý nghĩa khi có khả năng trả lời các câu hỏi toàn hệ thống dù dữ liệu nằm rải rác ở nhiều site. Trong đề án này, các truy vấn phân tán được hiện thực thông qua FastAPI coordinator, thay vì yêu cầu người dùng truy vấn từng database riêng lẻ. Điều này thể hiện ba bài học học thuật quan trọng: dữ liệu phân tán không đồng nghĩa với người dùng phải thao tác rời rạc; middleware có thể đóng vai trò lớp logic toàn cục; và bài toán phân tán không chỉ là lưu dữ liệu ở nhiều nơi mà còn là tổng hợp và ra quyết định từ nhiều nơi đó.

5.2. Sơ đồ xử lý truy vấn phân tán tổng quát

Mọi truy vấn phân tán đều tuân theo mẫu chung: frontend gửi một request, coordinator phân tán request đó tới từng site, thu gom kết quả từng phần rồi tổng hợp thành response thống nhất trả về frontend.



Hình 6. Kiểm tra sản phẩm còn ở những kho nào

5.3. Truy vấn tồn kho toàn hệ thống

Endpoint: GET /inventory/{sku}/global: truy vấn này cho biết một SKU hiện còn hàng tại site/kho nào, số lượng khả dụng (available_qty) là bao nhiêu và tổng reserved là bao nhiêu.

Dữ liệu sử dụng

- Bảng inventory tại cả 3 site north, central, south.
- Bảng warehouses để hiển thị tên kho và thông tin site.

Cách xử lý

1. Coordinator gửi truy vấn đồng thời tới cả 3 site.
2. Mỗi site đọc inventory cục bộ theo điều kiện WHERE sku = <sku_cần_tìm>.
3. Coordinator thu gom partial result từ 3 site.
4. Hợp nhất thành một response duy nhất, tính tổng available_qty và reserved_qty toàn hệ thống.

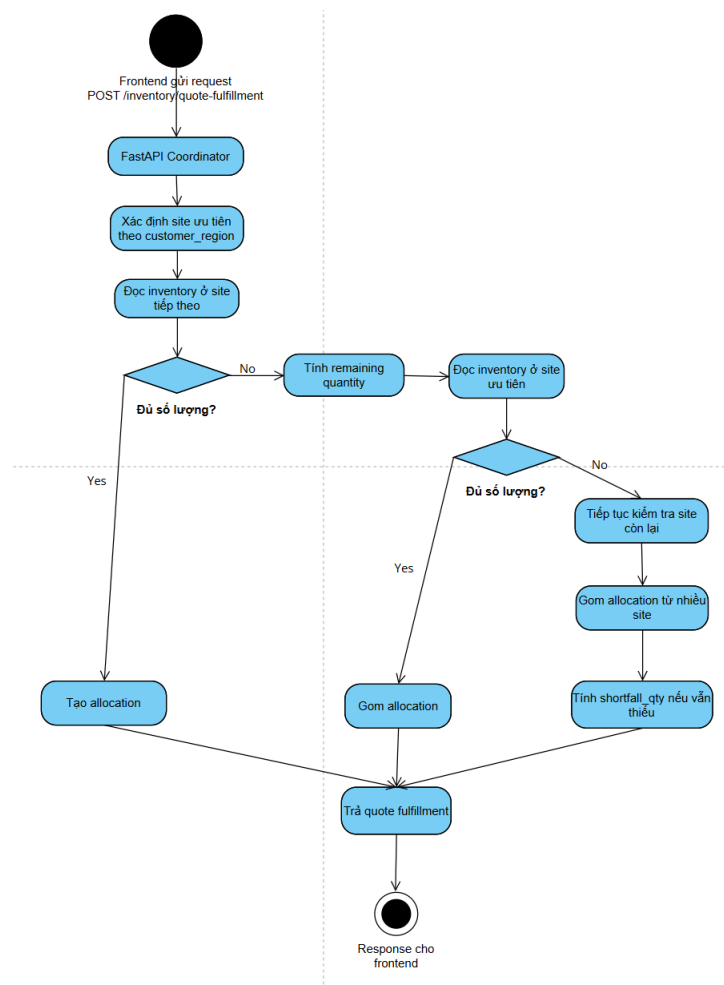
5. Ghép thêm thông tin tên kho và site từ bảng warehouses (đã nhân bản) để hiển thị đầy đủ.

Chi phí truyền dữ liệu

Coordinator gửi 3 request nhỏ (mỗi request chỉ truy vấn một SKU cụ thể). Mỗi site trả về rất ít dòng dữ liệu, không cần gửi toàn bộ bảng inventory. Đây là truy vấn có chi phí mạng thấp nhất trong các truy vấn phân tán của hệ thống.

Ý nghĩa phân tán và sơ đồ hoạt động

Đây là truy vấn tiêu biểu nhất để chứng minh dữ liệu vật lý phân tán nhưng người dùng vẫn nhìn thấy một ảnh logic thống nhất. Người dùng không cần biết hàng đang ở kho nào thuộc site nào – hệ thống tự động tổng hợp và trình bày.



Hình 7. Phân bổ đơn hàng theo vùng khách hàng

5.4. Truy vấn quote phân bổ đơn hàng

Endpoint: POST /inventory/quote-fulfillment: truy vấn này không chỉ đọc dữ liệu từ nhiều site mà còn ra quyết định điều phối dựa trên dữ liệu phân tán.

Cho biết nếu khách hàng đặt một SKU với số lượng nhất định, hệ thống sẽ ưu tiên lấy hàng từ site nào và lấy thêm từ site nào nếu site đầu tiên không đủ.

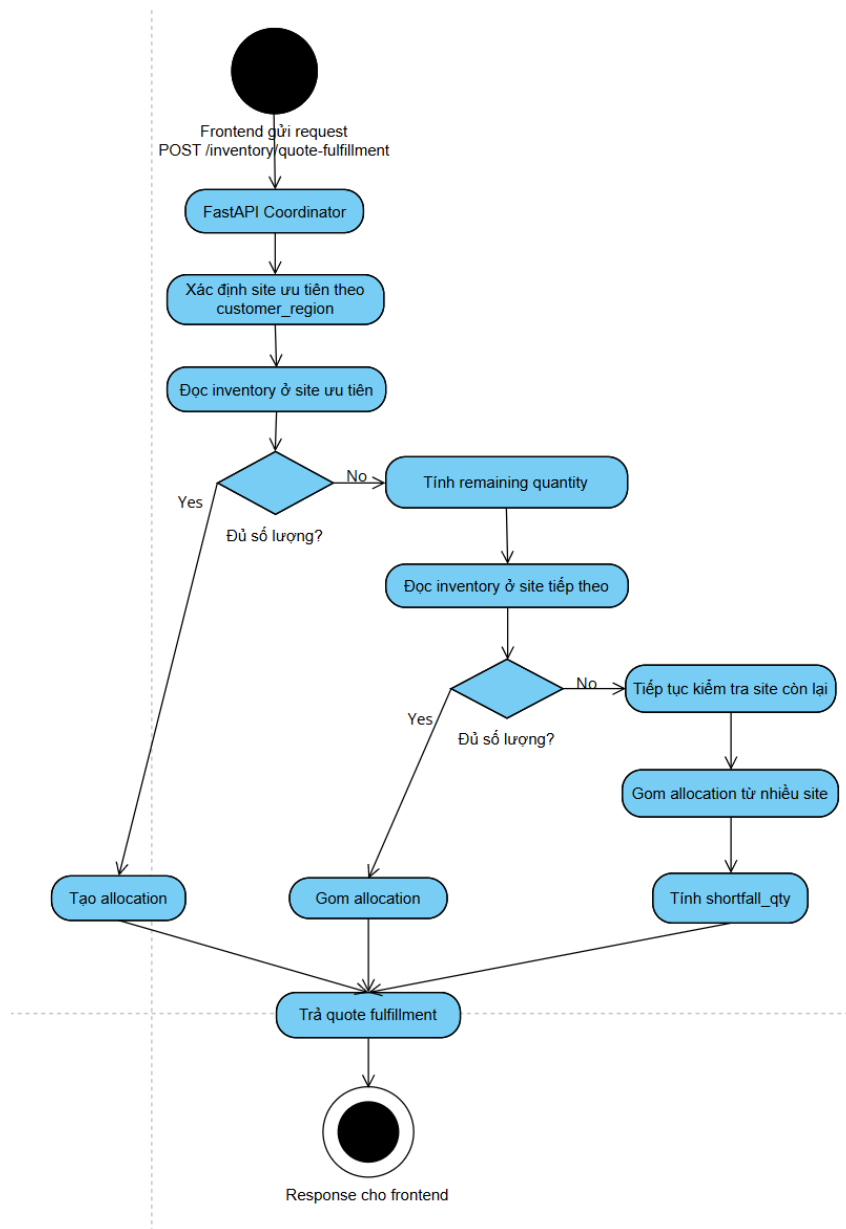
Dữ liệu đầu vào

Ví dụ request:

```
{  
  "sku": "LAP-01",  
  "quantity": 10,  
  "customer_region": "north"  
}
```

Cách xử lý

1. Xác định site ưu tiên theo customer_region (ví dụ: north → ưu tiên site north trước).
2. Đọc tồn kho tại site ưu tiên để kiểm tra số lượng khả dụng.
3. Nếu đủ số lượng: tạo allocation từ một site duy nhất.
4. Nếu thiếu: tính $\text{remaining_quantity} = \text{requested_qty} - \text{available_at_priority_site}$.
5. Đọc tiếp tồn kho tại các site theo thứ tự ưu tiên cho đến khi gom đủ.
6. Tạo danh sách allocations với từng site và số lượng tương ứng.
7. Trả về shortfall_qty nếu toàn hệ thống vẫn không đủ hàng.



Hình 8. Sơ đồ truy vấn quote phân bổ hàng

5.5. Truy vấn doanh thu theo site

Endpoint: GET /reports/revenue-by-site: tính doanh thu đã hoàn tất tại từng site và tổng cộng toàn hệ thống. Đây là ví dụ điển hình của bài toán tổng hợp dữ liệu phân mảnh ngang.

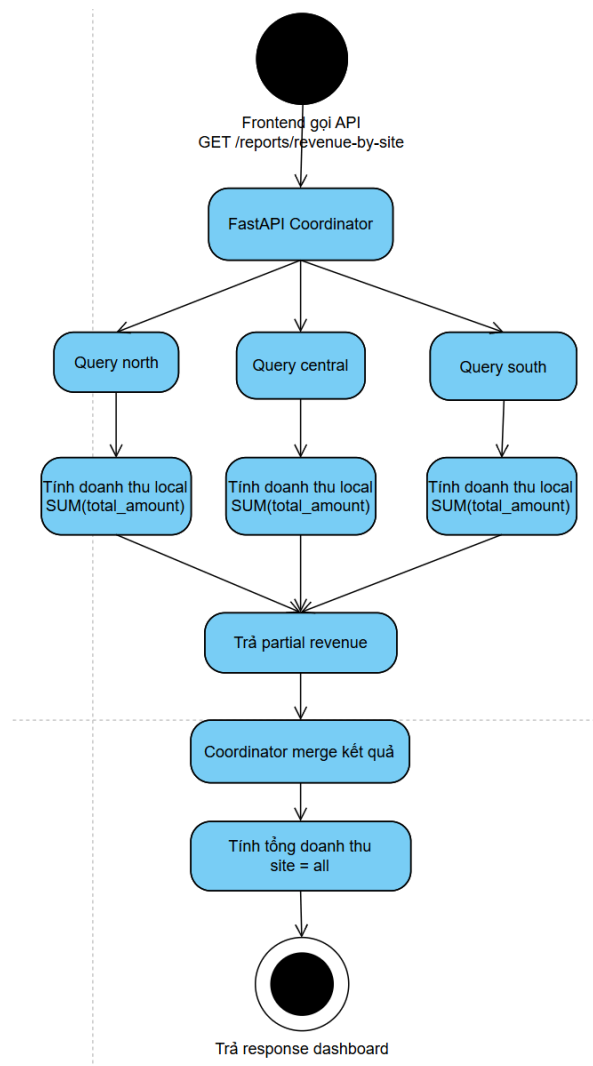
Dữ liệu sử dụng

- Bảng orders tại từng site, lọc theo status = 'completed'.

Cách xử lý và tối ưu

Thay vì kéo toàn bộ bảng orders về coordinator rồi xử lý, hệ thống áp dụng nguyên tắc xử lý cục bộ trước, tổng hợp sau: mỗi site tự tính

SUM(total_amount) và GROUP BY site ngay tại local database, coordinator chỉ nhận kết quả đã tổng hợp rồi cộng dồn thành dòng tổng site = all. Cách này giảm đáng kể lượng dữ liệu truyền qua mạng.



Hình 9. Sơ đồ truy vấn doanh thu theo site

5.6. Truy vấn top sản phẩm bán chạy

Endpoint: GET /reports/top-products: xác định SKU bán chạy nhất trên toàn hệ thống. Đây là ví dụ điển hình của truy vấn kiểu aggregate + merge giữa nhiều site.

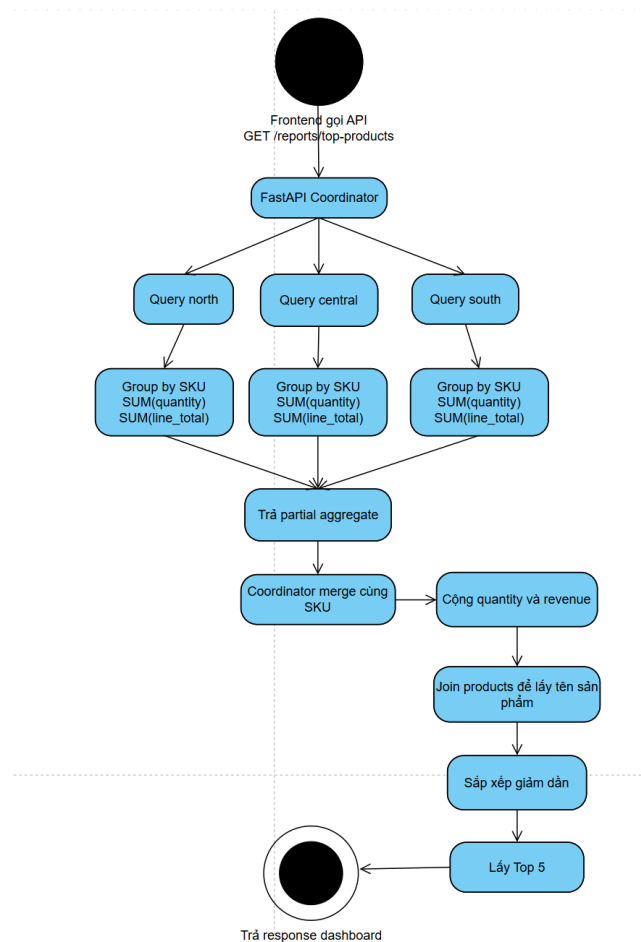
Dữ liệu sử dụng

- Bảng order_items tại cả 3 site để lấy thông tin số lượng bán và doanh thu.
- Bảng products (đã nhân bản) để ghép tên sản phẩm theo SKU.

Cách xử lý

1. Mỗi site tự thực hiện GROUP BY sku, tính SUM(quantity) và SUM(line_total) trên dữ liệu cục bộ.
2. Coordinator thu gom partial aggregate từ 3 site.
3. Gộp các nhóm cùng SKU: cộng dồn quantity và revenue từ nhiều site.
4. Join với bảng products để lấy tên sản phẩm (thực hiện local vì products đã được nhân bản).
5. Sắp xếp giảm dần theo quantity và lấy top 5 SKU.

Tối ưu: mỗi site aggregate trước khi gửi về nên coordinator chỉ nhận tối đa N dòng (N = số SKU khác nhau) thay vì toàn bộ bảng order_items.



Hình 10. Sơ đồ truy vấn top sản phẩm bán chạy

5.7. Truy vấn đơn hàng đa kho

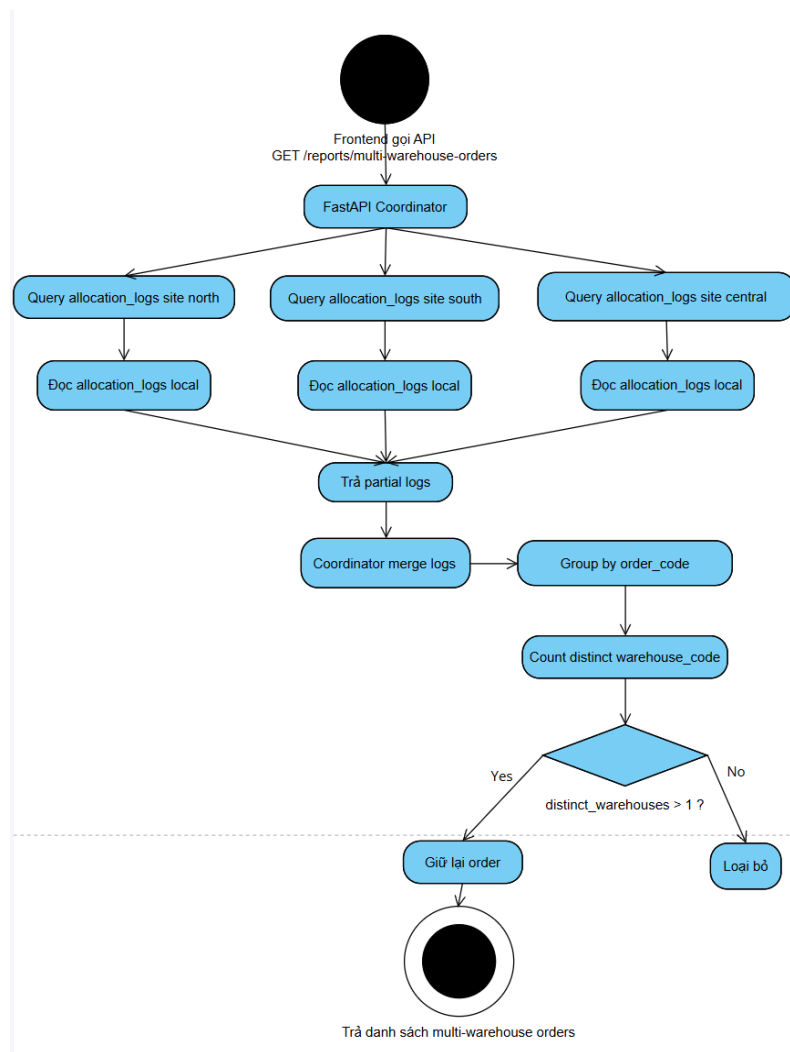
Endpoint: GET /reports/multi-warehouse-orders: tìm các đơn hàng đã được phân bổ từ nhiều warehouse/site khác nhau. Truy vấn này có giá trị lớn vì cho thấy split order là có thật và có thể đo lường được bằng dữ liệu.

Dữ liệu sử dụng

- Bảng allocation_logs tại tất cả các site.

Cách xử lý

1. Coordinator đọc allocation_logs từ tất cả các site.
2. Gộp toàn bộ logs lại và nhóm theo order_code.
3. Với mỗi order, đếm số warehouse_code phân biệt (COUNT DISTINCT warehouse_code).
4. Chỉ giữ lại các đơn hàng có distinct_warehouses > 1, tức là đơn hàng thực sự được cấp phát từ nhiều kho.



Hình 11. Sơ đồ truy vấn đơn hàng đa kho

5.8. Bảng tổng hợp truy vấn phân tán

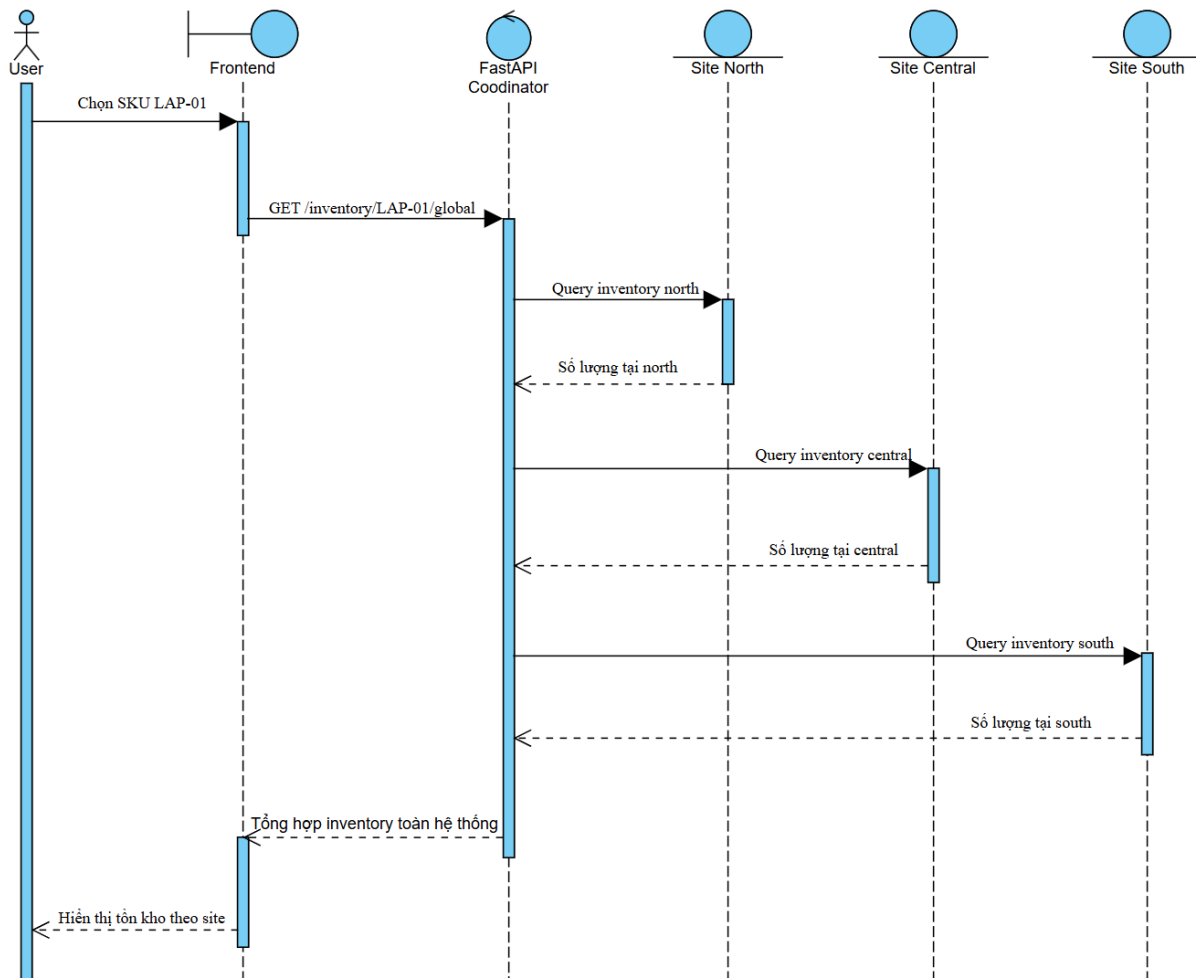
Truy vấn	Endpoint	Site tham gia	Kiểu xử lý	Kết quả chính

Inventory global	GET /inventory/{sku} /global	3 site	fan-out + aggregate	Tổng tồn và chi tiết từng kho
Quote fulfillment	POST /inventory/quote- fulfillment	2–3 site tùy SKU	fan-out + decision	Allocation theo site, shortfall_qty
Revenue by site	GET /reports/revenue- by-site	3 site	local aggregate + global merge	Doanh thu từng site và toàn hệ thống
Top products	GET /reports/top-prod ucts	3 site	group + merge	Top 5 SKU bán chạy nhất
Multi-ware house orders	GET /reports/multi-w arehouse-orders	Tất cả site có log	group + filter	Đơn hàng split nhiều kho

VI. Tình huống phân tán tiêu biểu

6.1. Tra cứu tồn kho toàn hệ thống

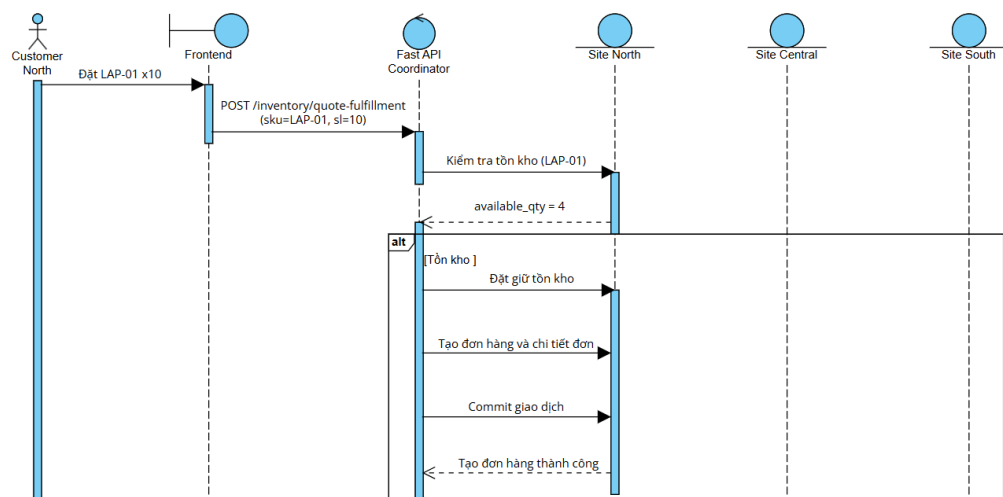
Người dùng nhập một SKU, ví dụ LAP-01. Middleware sẽ gửi truy vấn tới cả 3 site đồng thời, đọc tồn kho cục bộ từ từng site, tổng hợp kết quả về một phản hồi thống nhất và trả về tổng tồn, tổng reserved cùng chi tiết từng kho.

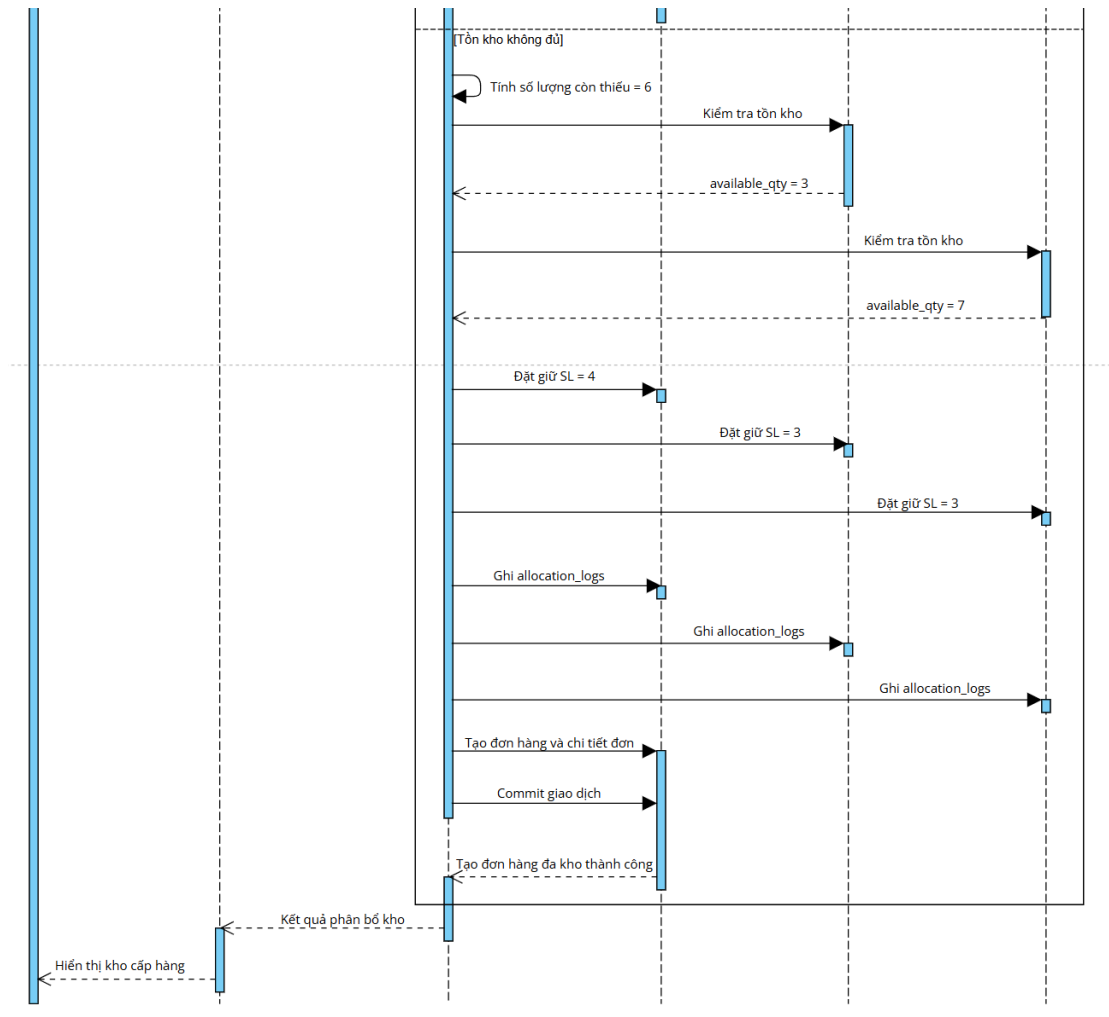


Hình 12. Sơ đồ tra cứu tồn kho toàn hệ thống

6.2. Đặt hàng khi kho gần nhất không đủ hàng

Đây là tình huống thể hiện rõ nhất cơ chế điều phối đa kho của hệ thống. Lấy ví dụ: khách ở miền Bắc đặt LAP-01 số lượng 10, nhưng kho miền Bắc chỉ còn 2 đơn vị, kho miền Trung còn 0, kho miền Nam còn 8.





Hình 13. Sơ đồ đặt hàng khi kho không đủ hàng

Bước 1: Tiếp nhận yêu cầu đặt hàng

Coordinator nhận request tạo đơn hàng với thông tin: customer thuộc miền Bắc, sku = LAP-01, quantity = 10. Do khách thuộc miền Bắc nên primary_site được xác định là north. Site north sẽ đóng vai trò xử lý chính của đơn hàng này.

Bước 2: Kiểm tra tồn kho tại site ưu tiên (north)

Coordinator gửi truy vấn tới site north để lấy available_qty của LAP-01. Kết quả trả về: available_qty = 2. Do $2 < 10$ nên site north không đủ khả năng đáp ứng toàn bộ đơn hàng.

Bước 3: Tính phần còn thiếu

Hệ thống tính: remaining = $10 - 2 = 8$ đơn vị. Đây là số lượng cần lấy bổ sung từ site khác.

Bước 4: Kiểm tra tồn kho ở các site còn lại

Coordinator gửi request tới central và south. Kết quả: central = 0, south = 8. Coordinator tổng hợp: north = 2, central = 0, south = 8, tổng = 10. Kết luận: đủ hàng để tạo đơn từ hai site north và south.

Bước 5: Reserve tồn kho tại từng site

Trước khi commit đơn hàng, hệ thống thực hiện reserve stock ở từng site để tránh race condition. Cụ thể: north reserve 2, south reserve 8. Trong bước này, câu lệnh SELECT ... FOR UPDATE được sử dụng để khóa row inventory đang cập nhật. Mục tiêu là tránh 2 request cùng lấy một lượng hàng, tránh âm tồn kho và đảm bảo nhất quán trong môi trường đồng thời.

Bước 6: Tạo order và order_items

Sau khi reserve thành công, coordinator tạo bản ghi orders và order_items. Đơn hàng vẫn thuộc primary_site = north, nhưng order_items sẽ chứa thông tin fulfillment theo nhiều warehouse: warehouse_north cấp 2, warehouse_central cấp 0, warehouse_south cấp 8.

Bước 7: Ghi allocation_logs

Hệ thống ghi log phân bổ tồn kho vào bảng allocation_logs với thông tin: order_id = ORD001, sku = LAP-01, north → 2, south → 8. Mục đích của bảng này là audit, debug và theo dõi split-order đa kho, giúp xác định rõ đơn nào được cấp phát từ những kho nào.

Bước 8: Commit giao dịch

Nếu mọi site reserve thành công, coordinator thực hiện commit. Kết quả: tồn kho ở north và south bị trừ tương ứng, bản ghi order được tạo thành công và allocation log được ghi đầy đủ.

Trường hợp lỗi

Nếu một site reserve thất bại trong quá trình xử lý (ví dụ: south hết hàng trong lúc xử lý do một request khác đặt trước), coordinator thực hiện release/rollback reserve ở các site đã giữ hàng trước đó. Điều này đảm bảo: không âm tồn kho, tránh giữ stock bị treo (không ai mua được nhưng order không tồn tại) và bảo toàn tính nhất quán của toàn hệ thống.

VII. Đồng thời và nhất quán tồn kho

7.1. Bài toán đặt ra

Trong hệ thống bán hàng đa kho, nhiều người dùng có thể đồng thời đặt cùng một sản phẩm. Đây là vấn đề điển hình của concurrency trong cơ sở dữ liệu. Nếu các transaction cùng đọc dữ liệu tồn kho trước khi có cơ chế khóa, hệ thống rất dễ gặp các lỗi nghiêm trọng:

- Mất cập nhật (lost update): hai transaction cùng đọc `available_qty = 5`, cùng trừ đi 5, cả hai cùng ghi lại 0 – nhưng thực tế đã bán 10 đơn vị.
- Ghi đè giao dịch của nhau (dirty write): một transaction ghi đè kết quả của transaction khác đang xử lý dở.
- Âm tồn kho (negative inventory): `available_qty` bị trừ quá số lượng thực có, dẫn tới tồn kho âm.
- Báo cáo sai sau đồng bộ: nếu dữ liệu tồn kho bị sai, mọi báo cáo tổng hợp từ nhiều site đều trở nên không tin cậy.

Vì vậy, mục tiêu bắt buộc của hệ thống là: tồn kho không bao giờ nhỏ hơn 0, một SKU không bị cấp phát vượt quá số lượng đang có, và request thất bại không để lại trạng thái dở dang.

7.2. Khái niệm nhất quán trong demo

Nhất quán ở đây được hiểu theo nghĩa thực dụng, cụ thể:

- Mỗi lần tạo đơn hàng phải bảo toàn ràng buộc `available_qty >= 0` ở mọi kho liên quan.
- Số lượng đã giữ chỗ phải được phản ánh chính xác trong `reserved_qty`, không được lớn hơn số lượng thực tế đang giữ.
- Nếu giao dịch không hoàn thành vì bất kỳ lý do gì, lượng đã giữ chỗ phải được trả lại đầy đủ.
- Log audit phải đủ để giải thích điều gì đã xảy ra với từng đơn vị tồn kho tại mọi thời điểm.

7.3. Mô hình `available_qty` và `reserved_qty`

Hệ thống không trừ tồn kho ngay khi khách hàng vừa bấm đặt hàng. Thay vào đó, bảng inventory sử dụng hai trường để phân tách trạng thái hàng hóa:

Trường	Ý nghĩa
available_qty	Số lượng còn có thể bán ngay. Đây là số lượng thực sự có thể đáp ứng cho các đơn hàng mới.
reserved_qty	Số lượng đã giữ chỗ bởi transaction đang xử lý nhưng chưa được commit thành đơn hàng hoàn chỉnh.

Lợi ích của mô hình này là tách bạch rõ ràng giữa "còn hàng vật lý" và "đã được transaction tạm chiếm", giúp mô phỏng logic reserve/commit/release gần với hệ thống thật. Khi khách hàng đặt LAP-01 x2 thì: available_qty giảm từ 10 xuống 8 và reserved_qty tăng từ 0 lên 2. Hàng đã được giữ cho request hiện tại, request khác không thể lấy phần hàng đó nữa, nhưng order vẫn chưa hoàn tất. Khi order được tạo thành công: reserved_qty giảm từ 2 về 0 và phần inventory chính thức được xem là đã xuất kho.

7.4. Cơ chế SELECT ... FOR UPDATE

Khi một request đặt hàng bắt đầu reserve tồn kho, hệ thống không đọc inventory theo kiểu thông thường mà sử dụng câu lệnh SELECT ... FOR UPDATE để khóa row inventory tương ứng ở mức row-level. Nếu transaction B đến sau muốn đọc-cập nhật cùng dòng theo cách tương tự, transaction B phải chờ cho đến khi transaction A hoàn tất (commit hoặc rollback).

Ví dụ: Khách A và khách B cùng lúc đặt LAP-01 x5 trong khi kho chỉ còn available_qty = 5.

Request A vào trước: thực hiện SELECT ... FOR UPDATE, row inventory bị khóa, reserve thành công: available_qty = 5 → 0, reserved_qty = 0 → 5.

Request B đến sau: cố gắng SELECT ... FOR UPDATE nhưng row đang bị khóa bởi request A nên phải chờ. Sau khi transaction A hoàn tất và row được mở khóa, request B đọc lại và thấy available_qty = 0. Request B bị từ chối do không đủ hàng.

Cơ chế này loại bỏ hoàn toàn tình trạng overselling – tồn kho âm hoặc bán vượt số lượng thật. Hệ thống sử dụng isolation level mặc định của PostgreSQL kết hợp với FOR UPDATE, không cần dùng SERIALIZABLE vì chi phí cao hơn và không cần thiết cho bài toán minh họa.

7.5. Chu trình reserve/commit/release

7.5.1 Reserve – Giữ chỗ tồn kho

- Khóa bản ghi tồn kho bằng SELECT ... FOR UPDATE tại site tương ứng.
- Kiểm tra điều kiện available_qty >= requested_qty. Nếu không đủ thì fail request ngay lập tức.
- Giảm available_qty đi một lượng bằng requested_qty.
- Tăng reserved_qty lên một lượng tương ứng.
- Ghi bản ghi vào bảng inventory_audit với action = 'reserve', lưu trạng thái before và after.

7.5.2 Commit – Xác nhận giao dịch thành công

- Sau khi order và order_items được tạo thành công ở tất cả các site liên quan.
- Giảm reserved_qty về 0 (hoặc giảm đúng lượng đã reserve).
- Ghi bản ghi allocation_logs xác nhận warehouse đã cấp bao nhiêu hàng cho đơn nào.
- Ghi bản ghi inventory_audit với action = 'commit'.

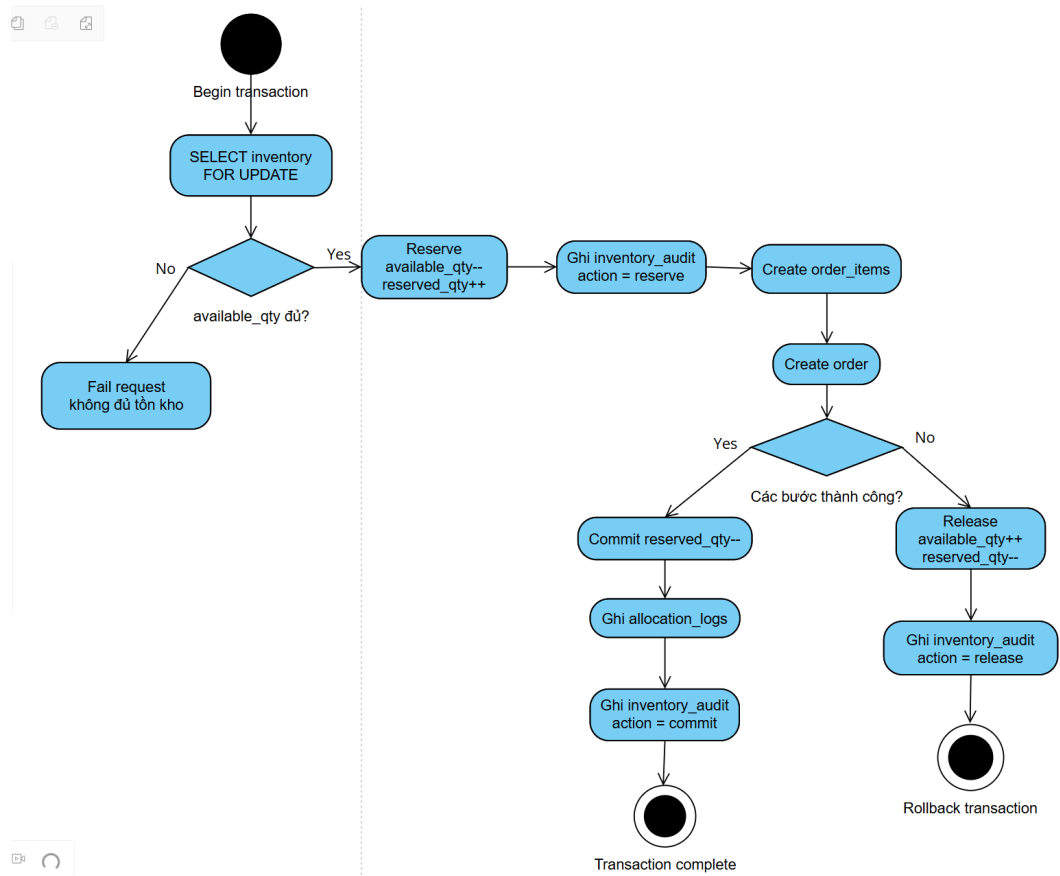
7.5.3 Release – Hoàn trả khi giao dịch thất bại

Trong quá trình xử lý đa kho, có thể xảy ra lỗi ở giữa chừng. Ví dụ: khách đặt LAP-01 x10, đã reserve north = 2 thành công nhưng south reserve thất bại. Nếu không xử lý rollback, north vẫn bị giữ hàng dẫn tới stock bị treo (không ai mua được nhưng order không tồn tại). Do đó khi xảy ra lỗi ở bất kỳ bước nào:

- Tăng lại available_qty về giá trị ban đầu ở tất cả các site đã reserve.
- Giảm reserved_qty về 0 ở các site tương ứng.
- Ghi bản ghi inventory_audit với action = 'release'.
- Rollback toàn bộ transaction.

→ Kết quả: toàn bộ tồn kho trở về trạng thái trước khi request được xử lý, không có dữ liệu bị để lại ở trạng thái dở dang.

7.5.4 Sơ đồ chu trình reserve/commit/release



Hình 14. Sơ đồ chu trình reserve/commit/release

7.6. Kịch bản đồng thời trong bản demo

Endpoint demo: POST /orders/demo-concurrency: hai khách hàng (ví dụ CUS-N-02 và CUS-N-03) cùng đặt SKU PHN-01 với quantity = 6 trong gần như cùng một thời điểm. Hệ thống sẽ xử lý theo cơ chế SELECT ... FOR UPDATE.

Trường hợp 1: Tồn kho toàn hệ thống vẫn đủ

Ví dụ: available_qty toàn hệ thống = 20. Hai khách cùng đặt $6 + 6 = 12$. Request 1 lock row, reserve 6, unlock. Request 2 sau đó lock row, đọc lại available_qty = 14, đủ để reserve thêm 6. Cả hai request đều thành công và tồn kho còn lại 8 đơn vị.

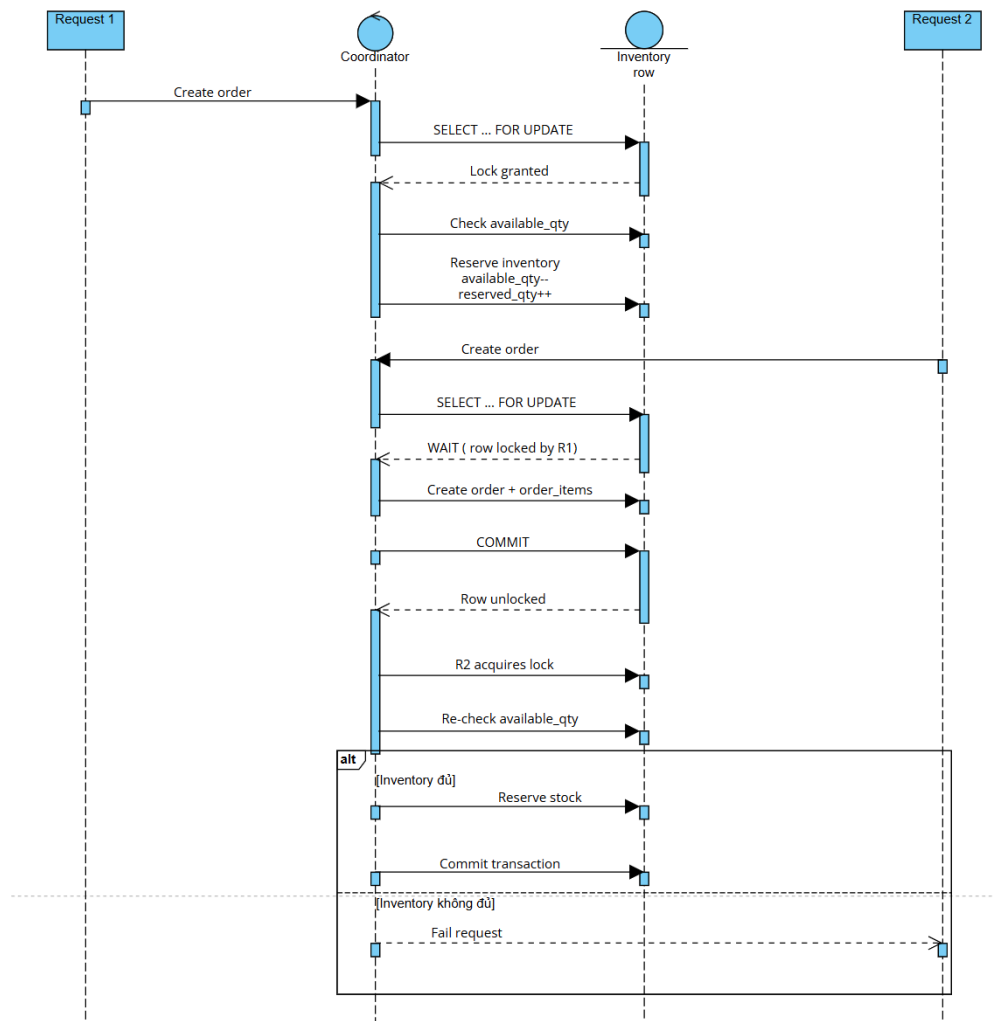
Trường hợp 2: Tồn kho không còn đủ

Ví dụ: available_qty toàn hệ thống = 8. Request 1 lock row, reserve 6, tồn còn 2. Request 2 sau khi lock row đọc lại available_qty = 2, nhỏ hơn requested_qty = 6, bị fail request. Hệ thống trả về lỗi không đủ tồn kho cho request 2.

- Bất biến của hệ thống: trong mọi trường hợp, hệ thống đảm bảo các bất

biến sau:

- + Không xảy ra overselling: không request nào được phép cấp phát nhiều hơn `available_qty` tại thời điểm đó.
- + `available_qty` không bao giờ âm tại bất kỳ kho nào.
- + Tổng tồn kho cuối cùng luôn nhất quán và có thể giải thích được qua `inventory_audit`.
- + Không có stock bị treo nhờ cơ chế `reserve/release` được áp dụng ngay cả khi có lỗi.



Hình 15. Sơ đồ tuần tự concurrency

7.7. Vai trò của bảng `inventory_audit`

Bảng `inventory_audit` ghi lại mọi thay đổi tồn kho dưới dạng append-only log. Mỗi bản ghi lưu: `request_code` (định danh request gây ra thay đổi), `sku` và `warehouse_code` xác định hàng nào ở kho nào, `action` (`reserve/commit/release`),

delta_available và delta_reserved (độ thay đổi tương ứng), và created_at để xác định thứ tự thời gian.

Ví dụ log inventory_audit cho một giao dịch thành công:

SKU	Warehouse	Action	delta_available	delta_reserved
LAP-01	HN-01	reserve	-2	+2
LAP-01	HCM-01	reserve	-8	+8
LAP-01	HN-01	commit	0	-2
LAP-01	HCM-01	commit	0	-8

Ví dụ log khi giao dịch bị rollback:

SKU	Warehouse	Action	delta_available	delta_reserved
LAP-01	HN-01	reserve	-2	+2
LAP-01	HCM-01	reserve	-8	+8
LAP-01	HN-01	release	+2	-2
LAP-01	HCM-01	release	+8	-8

Nhờ bảng inventory_audit, hệ thống có thể xác định: request nào đã reserve, commit hoặc release; thứ tự thay đổi dữ liệu theo thời gian; có hay không tồn tại trạng thái bất nhất quán. Điều này đặc biệt quan trọng trong hệ phân tán vì nhiều site có thể cùng tham gia xử lý một đơn hàng và cần có log toàn diện để audit và debug.

7.8. Mức độ hoàn chỉnh của giải pháp

Bản demo hiện tại không triển khai distributed transaction manager hoàn chỉnh kiểu Two-Phase Commit (2PC) thực thụ. Tuy nhiên, giải pháp hiện tại đủ đáp ứng yêu cầu mô phỏng tính đồng thời và nhất quán trong môi trường phân tán vì:

- Có khóa mức dòng (row-level lock) thông qua SELECT ... FOR UPDATE.

- Có cơ chế reserve/commit/release rõ ràng và tách biệt.
- Có audit log đầy đủ để chứng minh tính đúng đắn của từng transaction.
- Có thể chứng minh không âm kho bằng demo thực tế với dữ liệu thật.

VIII. Cài đặt hệ thống

8.1. Cài đặt hạ tầng web

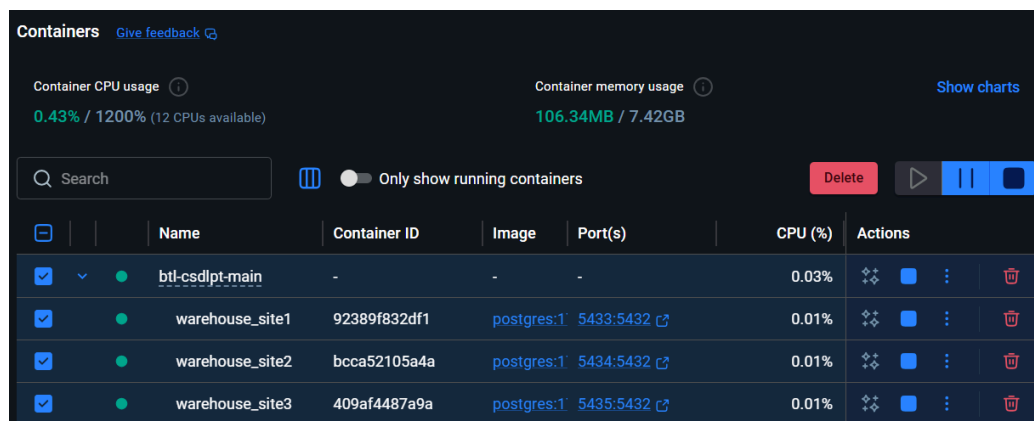
Hạ tầng web bao gồm ba thành phần khởi động theo thứ tự: cụm database, server API và giao diện người dùng

8.1.1. Khởi động cụm database

Ba container PostgreSQL 17 được khởi động đồng thời qua Docker Compose. Mỗi container tương ứng với một site địa lý, lắng nghe trên cổng riêng biệt:

Container	Site	Cổng	Database
warehouse_site1	north	5433	warehouse_north
warehouse_site2	central	5434	warehouse_central
warehouse_site3	south	5435	warehouse_south

Dùng lệnh để khởi chạy container đã cài trước: `docker compose up -d`



Hình 16. Khởi chạy thành công ba container

8.1.2. Khởi động FastAPI

Sau khi cài đặt gói Python, server API được khởi động và kết nối tới cả ba database theo chuỗi DSN cấu hình trong file `.env`

Chạy lệnh để tải các thư viện cần: `pip install -r requirements.txt`

Chạy lệnh để chạy API: `uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload`

```
C:\Users\Admin\Downloads\BTL-CSDLPT-main\BTL-CSDLPT-main>uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\Admin\\Downloads\\BTL-CSDLPT-main\\BTL-CSDLPT-main']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [19840] using StatReload
INFO: Started server process [27296]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Hình 17. Khởi chạy thành công API

8.1.3. Khởi động frontend

Chạy lệnh để cài lệnh “npm” và khởi chạy frontend: `cd frontend && npm install && npm run dev`

```
C:\Users\Admin\Downloads\BTL-CSDLPT-main\BTL-CSDLPT-main\frontend>npm run dev

> frontend@0.0.0 dev
> vite

VITE v8.0.9 ready in 186 ms

➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help
```

Hình 18. Khởi chạy thành công frontend

Giao diện phục vụ tại `http://127.0.0.1:5173`, giao tiếp với FastAPI hoàn toàn qua HTTP REST API.

8.1.4. Khởi tạo dữ liệu mẫu

Sau khi cả ba tầng hoạt động, gọi endpoint seed để tạo schema và nạp dữ liệu ban đầu vào cả ba site:

Chạy lệnh: `curl -X POST http://127.0.0.1:8000/demo/seed`

```
C:\Users\Admin\Downloads\BTL-CSDLPT-main\BTL-CSDLPT-main>curl -X POST http://127.0.0.1:8000/demo/seed
{"status":"ok"}
```

Hình 19. Tạo dữ liệu thành công cho 3 site

Lưu ý: lúc này 3 site phải đang chạy và API đã khởi động

Thao tác này thực thi tuần tự hai file sql trên từng site: 01-schema.sql và 02-seed.sql theo chiến lược phân tán đã thiết kế

8.2. Cài đặt logic phân tán

Logic phân tán được thể hiện qua cách dữ liệu được tổ chức, cách coordinator kết nối tới nhiều site, và cách các cơ chế nhất quán được kích hoạt trong từng luồng nghiệp vụ

8.2.1. Phân tán dữ liệu qua seed khác nhau trên từng site

Ba site dùng cùng một schema, nhưng dữ liệu được seed khác nhau theo từng file 02-seed.sql riêng. Phần này là phần phân mảnh ngang và nhân bản được thực hiện:

- Dữ liệu nhân bản - categories, products, warehouses - được seed với nội dung hoàn toàn giống nhau trên cả ba site. Điều này đảm bảo mỗi site đều hiển thị tên sản phẩm, loại danh mục và thông tin kho mà không cần truy vấn xuyên site:

Ví dụ: Ở cả ba site đều nhân bản dữ liệu như dưới

```
INSERT INTO products (sku, name, price) VALUES
```

```
('LAP-01', 'Laptop Pro 14', 32000000),
```

```
('PHN-01', 'Phone X', 18000000), ...
```

- Dữ liệu phân mảnh - customers, inventory, order và các log - được seed khác nhau: mỗi site chỉ chứa dữ liệu thuộc vùng địa lý của mình. Cụ thể, site north chứa khách hàng miền Bắc với region_code = 'north' và tồn kho kho Hà Nội; site south chứa khách miền Nam và tồn kho kho TP.HCM. Điều kiện lọc này chính là biểu thức phân mảnh ngang đã làm ở phần lý thuyết:

```
-- site1/02-seed.sql (north)
```

```
INSERT INTO inventory VALUES ('WH-NORTH-01', 'LAP-01', 2, 0, 2);
```

```
-- site3/02-seed.sql (south)
```

```
INSERT INTO inventory VALUES ('WH-SOUTH-01', 'LAP-01', 8, 0, 2);
```

```
-- site2/02-seed.sql (central) -- LAP-01 có 0 đơn vị
```

```
INSERT INTO inventory VALUES ('WH-CENTRAL-01', 'LAP-01', 0, 0, 2);
```

8.2.2. Coordinator kết nối đồng thời ba site

File app/core/dbrouter.py là tầng điều phối. DatabaseRouter đọc ba DSN từ biến môi trường kết nối tới cả ba site. Mọi truy vấn phân tán đều đi qua lớp này:

```
# app/core/dbrouter.py
class DatabaseRouter:
    def __init__(self):
        self.sites = {site.code: site for site in allsites()}

    def executequery(self, site_code, query, params, ...):
        # Gửi truy vấn tới đúng site theo site_code
        with psycopg.connect(self.sites[site_code].dsn) as conn:
            ...
```

File app/core/siteregistry.py định nghĩa thứ tự ưu tiên site theo vùng khách hàng - đây là nơi chiến lược locality of reference được cấu hình. Khi khách hàng thuộc vùng north, hệ thống luôn hỏi site north trước rồi mới sang central, rồi south:

```
# app/core/siteregistry.py

REGION_ORDER = {
    'north': ['north', 'central', 'south'],

    'central': ['central', 'north', 'south'],

    'south': ['south', 'central', 'north'],
}
```

8.2.3. Truy vấn phân tán fan-out

Mỗi khi người dùng tra cứu tồn kho toàn hệ thống, inventoryservice.py gửi truy vấn lần lượt tới cả ba site và tổng kết quả. Người dùng chỉ thấy một response thống nhất, không biết dữ liệu đến từ ba database khác nhau:

```
# app/services/inventoryservice.py
for site in allsites():      # lặp qua north, central, south

    rows = router.executequery(site.code, query, {'sku': sku})
```


for row in rows:

```
items.append(row)      # gom kết quả từng site
```

```
total_available_qty += row['available_qty']
```

Tương tự, các báo cáo doanh thu và top sản phẩm áp dụng nguyên tắc xử lý cục bộ trước, tổng hợp sau: mỗi site tự tính SUM, GROUP BY trên dữ liệu của mình, coordinator chỉ nhận kết quả đã rút gọn rồi cộng dồn. Cách này làm giảm đáng kể lượng dữ liệu truyền qua mạng so với việc kéo toàn bộ bảng về xử lý

8.2.4. Cơ chế nhất quán tồn kho trong môi trường đồng thời

Phần logic phức tạp nhất và quan trọng nhất. Khi tạo đơn hàng transactioncoordinator.py thực hiện chu trình ba bước đảm bảo tính nhất quán dữ liệu ngay cả khi nhiều request cùng lúc:

- Bước Reserve: trước khi trừ tồn kho, hệ thống khóa dòng inventory bằng SELECT ... FOR UPDATE. Câu lệnh này ngăn mọi transaction khác đọc và cập nhật cùng dòng cho đến khi transaction hiện tại hoàn tất. Sau đó giảm available_qty và tăng reserved_qty để đánh dấu hàng đã được giữ chỗ:

```
# app/services/transactioncoordinator.py
```

```
SELECT available_qty FROM inventory
```

```
WHERE warehouse_code = %s AND sku = %s
```

```
FOR UPDATE;      -- khóa dòng, buộc request khác phải chờ
```

```
UPDATE inventory SET
```

```
available_qty = available_qty - %(qty)s,
```

```
reserved_qty = reserved_qty + %(qty)s
```

```
WHERE warehouse_code = %s AND sku = %s;
```

- Bước Commit: sau khi tạo bản ghi orders và order_items thành công trên primary site, hệ thống ghi allocation_logs xác nhận từng kho đã cấp bao nhiêu hàng, đồng thời giảm reserved_qty về 0. Đây là thời điểm hàng chính thức được xem là đã xuất kho

- Bước Release (rollback): nếu bất kỳ bước nào thất bại - ví dụ reserve thành công ở site north nhưng site south hết hàng trong lúc xử lý - coordinator lập tức hoàn trả available_qty về giá trị ban đầu ở tất cả các site đã reserve, đồng thời ghi audit log với action release. Không có dữ liệu nào bị để lại ở trạng thái giữa chừng:

```
# app/services/orderservice.py -- xử lý rollback
```

except Exception:

```
for reservation in reversed(reservations):
```

```
    transactioncoordinator.releasereservation(
```

```
        reservation, request_code, connection
```

```
    )
```

```
for conn in owned_connections.values():
```

```
    conn.rollback()
```

```
raise
```

8.2.5. Audit log phân tán

Mỗi thao tác thay đổi tồn kho - dù là reserve, commit hay release - đều được ghi một bản ghi vào bảng inventory_audit tại đúng site nơi diễn ra thao tác. Bảng này lưu giữ trạng thái delta (delta_available, delta_reserved) thay vì giá trị tuyệt đối, cho phép tái tạo lại toàn bộ lịch sử thay đổi tồn kho của bất kỳ SKU nào tại bất kỳ kho nào. Đây là cơ sở để chứng minh tính nhất quán trong môi trường phân tán

IX. Đánh giá hệ thống

9.1. Kết quả kiểm thử thực tế

Sau khi hoàn tất cài đặt, nhóm tiến hành kiểm thử theo kịch bản demo chuẩn. Toàn bộ kịch bản đều cho kết quả đúng như thiết kế

9.1.1. Kiểm tra health 3 site

Gọi GET /demo/health/distributed, hệ thống trả về trạng thái của cả ba site trong một response duy nhất. Ba node đều ở trạng thái “is_healthy: true” kèm tên database và thời điểm kiểm tra, xác nhận ba container PostgreSQL hoạt động song song và coordinator kết nối thành công tới cả ba.

Sử dụng lệnh: curl -X GET http://127.0.0.1:8000/demo/health/distributed

```
C:\Users\Admin\Downloads\BTL-CSDLPT-main\BTL-CSDLPT-main>curl -X GET http://127.0.0.1:8000/demo/health/distributed
[{"site_code":"north","site_name":"Kho miền Bắc","is_healthy":true,"database_name":"warehouse_north","checked_at":"2026-05-18T05:28:05.253326+00:00"}, {"site_code":"central","site_name":"Kho miền Trung","is_healthy":true,"database_name":"warehouse_central","checked_at":"2026-05-18T05:28:05.265463+00:00"}, {"site_code":"south","site_name":"Kho miền Nam","is_healthy":true,"database_name":"warehouse_south","checked_at":"2026-05-18T05:28:05.293801+00:00"}]
```

Hình 20. Kiểm tra health

9.1.2. Tra cứu tồn kho toàn hệ thống (fan-out)

Tra cứu SKU LAP-01 qua GET /inventory/LAP-01/global. Coordinator gửi truy vấn đồng thời tới ba site và trả về một response tổng hợp: north còn 2 đơn vị, central còn 0, south còn 8, tổng available_qty toàn hệ thống là 10. Đây là minh chứng rõ nhất cho tính trong suốt phân mảnh: người dùng thấy một mảnh dữ liệu thống nhất dù dữ liệu vật lý nằm ở ba database khác nhau

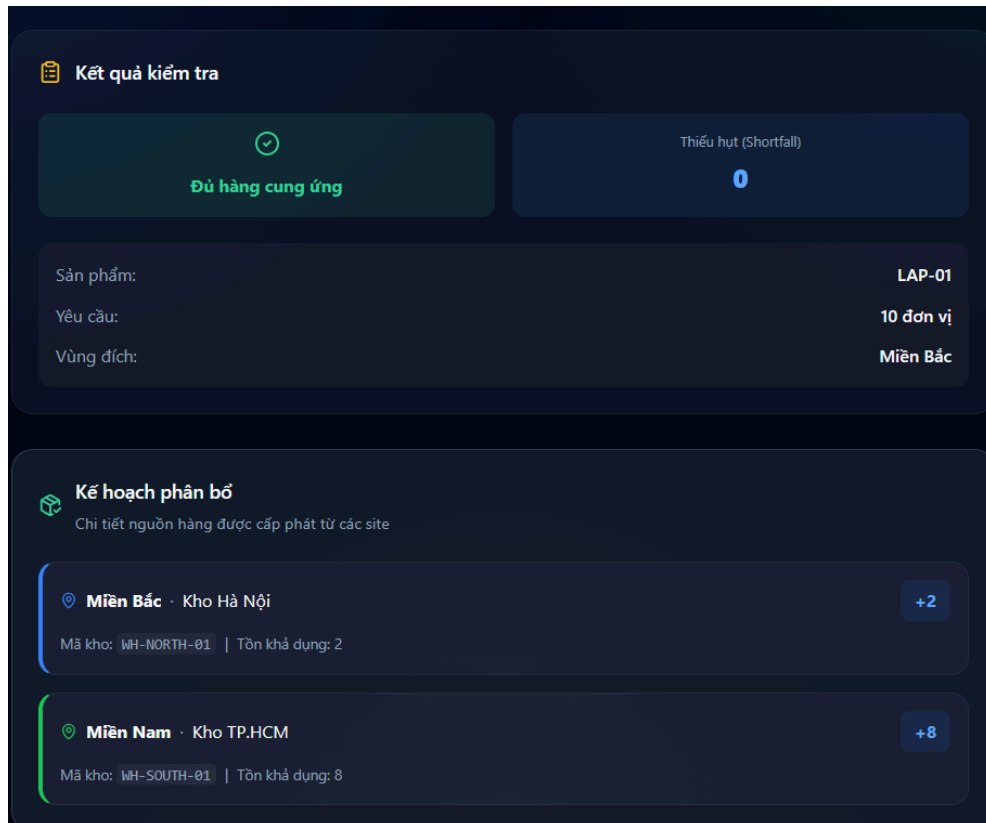
Chạy lệnh: curl -X GET http://127.0.0.1:8000/inventory/LAP-01/global

```
C:\Users\Admin\Downloads\BTL-CSDLPT-main\BTL-CSDLPT-main>curl -X GET http://127.0.0.1:8000/inventory/LAP-01/global
{"sku":"LAP-01","product_name":"Laptop Pro 14","total_available_qty":10,"total_reserved_qty":0,"items":[{"site_code":"north","warehouse_code":"WH-NORTH-01","warehouse_name":"Kho Hà Nội","city":"Hà Nội","available_qty":2,"reserved_qty":0,"reorder_level":2}, {"site_code":"central","warehouse_code":"WH-CENTRAL-01","warehouse_name":"Kho Đà Nẵng","city":"Đà Nẵng","available_qty":0,"reserved_qty":0,"reorder_level":2}, {"site_code":"south","warehouse_code":"WH-SOUTH-01","warehouse_name":"Kho TP.HCM","city":"TP.HCM","available_qty":8,"reserved_qty":0,"reorder_level":2}]}
```

Hình 21. Kiểm tra số lượng hàng mã LAP-01 cả hệ thống

9.1.3. Quote phân bổ theo vùng địa lý (locality of reference)

Gọi POST /inventory/quote-fulfillment với SKU LAP-01, số lượng 10, vùng north. Thuật toán lấy đủ 2 đơn vị từ site north trước , phần còn thiếu 8 đơn vị lấy từ site south. Site central không có hàng nên bị bỏ qua. Response trả về “is_fulfillable: true”, “shortfall)qty_: 0” và hai allocation từ hai site khác nhau



Hình 22. Phân bổ thành công

9.1.4. Tạo đơn hàng split order (điều phối đa site)

Gọi POST /order với CUS-N-01, LAP-01, số lượng 10, vùng north. Hệ thống thực hiện đầy đủ chu trình: reserve bằng SELECT FOR UPDATE ở cả north và south, tạo orders/order_items tại site north (primary site), ghi allocation_logs tại từng site , commit và trả về order_code mới với status completed. Tra lại tồn kho sau đó xác nhận available_qty tại north giảm từ 2 xuống 0. Không có sai lệch giữa các site

Tạo đơn hàng

Preset gợi ý: CUS-N-01 · LAP-01 · 10 · north

Customer code

CUS-N-01

SKU

LAP-01

Số lượng

10

Region

north

Tạo order

Order result

ORD-15D8294F

Primary site: north

COMPLETED

Total amount: 320.000.000 đ

Phân bổ của order vừa tạo

Chỉ tiết nguồn hàng được cấp phát từ các site

Miền Bắc · Kho Hà Nội

+2

Mã kho: WH-NORTH-01 | Tổng khả dụng: 2

Miền Nam · Kho TP.HCM

+8

Mã kho: WH-SOUTH-01 | Tổng khả dụng: 8

Chi tiết order

Lấy lại chi tiết từ backend theo order_code

ORD-15D8294F

Xem detail

Allocation timeline

Theo dõi các bước commit phân bổ đơn hàng giữa các site

Miền Bắc · WH-NORTH-01

COMMIT

SKU LAP-01 · Số lượng 2 · 05:51 18/5/26

Miền Nam · WH-SOUTH-01

COMMIT

SKU LAP-01 · Số lượng 8 · 05:51 18/5/26

Hình 23. Tạo đơn hàng LAP-01 cho khách CUS-N-01 thành công

9.1.5. Xem allocation timeline (audit log)

Gọi GET /orders/{order_code}. Response trả về danh sách allocation_logs cho thấy WH-NORTH-01 cấp 2 đơn vị và WH-SOUTH-01 cấp 8 đơn vị, kèm timestamp và action commit. Điều này khẳng định split order đa kho có thể audit toàn bộ lịch sử từng thao tác

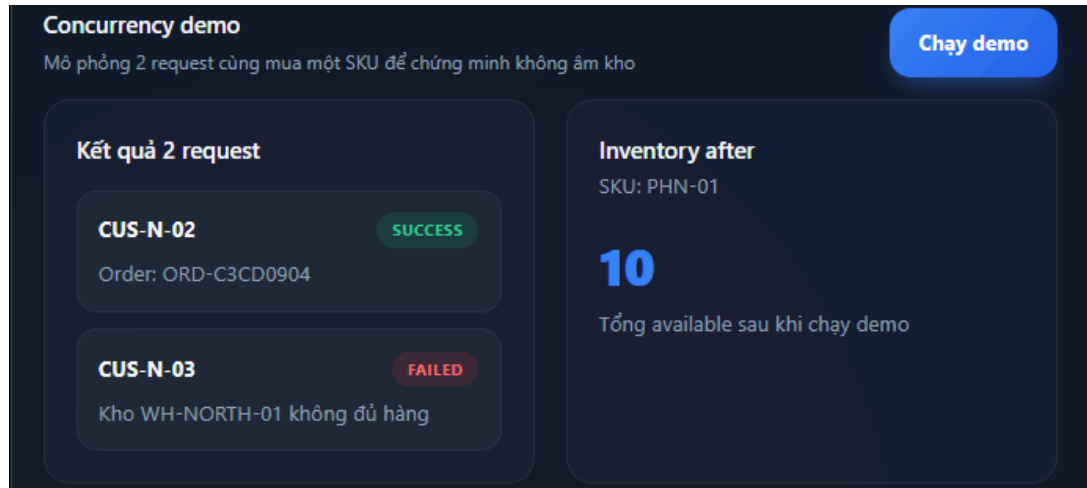
```
C:\Users\Admin\Downloads\BTL-CSDLPT-main\BTL-CSDLPT-main>curl -X GET http://127.0.0.1:8080/orders/ORD-15D8294F
{"order_code":"ORD-15D8294F","customer_code":"CUS-N-01","primary_site_code":"north","primary_warehouse_code":"WH-NORTH-01","status":"completed","total_amount":"320000000.00","created_at":"2026-05-18T05:51:43.474877","allocations":[{"order_code":"ORD-15D8294F","sku":"LAP-01","site_code":"north","warehouse_code":"WH-NORTH-01","allocated_qty":2,"action":"commit","created_at":"2026-05-18T05:51:43.474877"}, {"order_code":"ORD-15D8294F","sku":"LAP-01","site_code":"south","warehouse_code":"WH-SOUTH-01","allocated_qty":8,"action":"commit","created_at":"2026-05-18T05:51:43.492352"}]}
```

Hình 24. Kết quả allocation_logs của mã ORD-15D8294F

9.1.6. Demo đồng thời (nhất quán tồn kho)

Gọi POST /orders/demo-concurrency với SKU PHN-01, số lượng 6 mỗi đơn, hai khách hàng CUS-N-02 và CUS-N-03. Khi tổng tồn kho PHN-01 là 10 đơn vị, cơ chế SELECT FOR UPDATE đảm bảo request đến trước giữ được

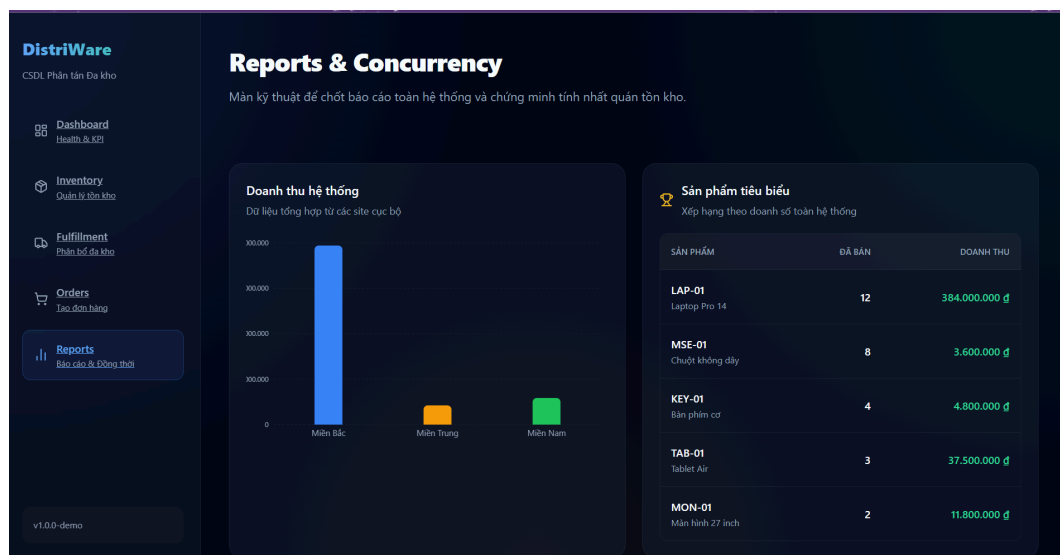
lock và reserve thành công, request đến sau phải chờ; khi đọc thấy available_qty còn 4, nhỏ hơn yêu cầu bị từ chối. Kết quả: một đơn is_success: true, một đơn is_success: false. Tổng available_qty sau demo không âm - overselling không xảy ra



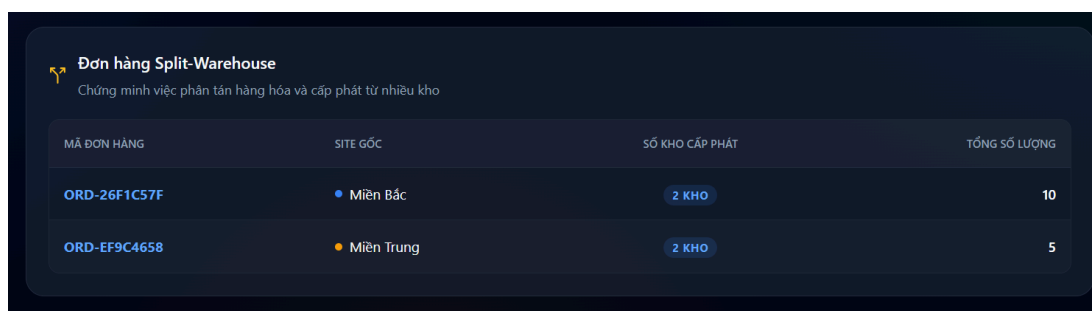
Hình 25. Kết quả demo chống âm kho

9.1.7. Báo cáo tổng hợp đa site

Các endpoint báo cáo tổng hợp chính xác dữ liệu phân tán. Revenue by site tính doanh thu từng site và dòng tổng all. Top products gộp order_items từ ba site theo nguyên tắc cục bộ trước, coordinator chỉ tổng hợp và sắp xếp kết quả đã rút gọn. Multi-warehouse orders liệt kê đơn hàng vừa tạo $\text{distinct_warehouse} = 2$, xác nhận split order là có thật và đo lường được.



Hình 26. Giao diện trang báo cáo tổng hợp doanh thu



MÃ ĐƠN HÀNG	SITE GỐC	SỐ KHO CẤP PHÁT	TỔNG SỐ LƯỢNG
ORD-26F1C57F	• Miền Bắc	2 KHO	10
ORD-EF9C4658	• Miền Trung	2 KHO	5

Hình 27. Đơn hàng liên kho

9.2. Kết quả kiểm thử tự động

Tên test	Luồng kiểm tra	Kết quả
test_quote_split_allocation	Quote LAP-01 x10 từ vùng north: is_fulfillable=true, shortfall=0, tổng allocated=10	PASS
test_quote_uses_multiple_sites	Allocations chứa đúng site {north, south}, không chứa central (hết hàng)	PASS
test_low_stock_summary_shape	Endpoint low-stock trả về list, không lỗi, cấu trúc dữ liệu hợp lệ	PASS

9.3. Đánh giá mức độ đáp ứng mục tiêu phân tán

Đặc trưng phân tán	Cơ chế hiện thực	Kết quả kiểm thử
Phân mảnh ngang	Seed dữ liệu theo region_code và site_code trên từng site riêng	Đạt, mỗi site chỉ lưu dữ liệu vùng mình
Nhân bản dữ liệu dùng chung	categories/products/warehouses seed giống nhau ở 3 site	Đạt, không cần join xuyên site khi hiển thị
Tính trong suốt vị trí	Người dùng chỉ thấy 1 API, không biết dữ liệu ở site nào	Đạt, xác nhận qua toàn bộ luồng demo

Tính trong suốt phân mảnh	Response tổng hợp từ nhiều site trả về như một khối thống nhất	Đạt, inventory global revenue, top products
Truy vấn fan-out	Coordinator gửi song song tới N site, gom kết quả	Đạt, latency thấp nhờ xử lý đồng thời
Tính nhất quán đồng thời	SELECT FOR UPDATE + reserve/commit/release ngăn overselling	Đạt, demo concurrency không âm kho
Audit log đầy đủ	Inventory_audit ghi mọi thao tác delta tại đúng site	Đạt, có thể tái tạo lại lịch sử
Rollback phân tán	Release reserve ở tất cả site đã giữ hàng khi có lỗi	Đạt, không có stock bị treo

9.4. Phân tích lý thuyết và hạn chế

Hệ thống demo đạt được mức nhất quán thực dụng phù hợp phạm vi đề tài. Cơ chế SELECT FOR UPDATE cung cấp row-level locking đủ mạnh để ngăn race condition trong môi trường đơn node coordinator. Tuy nhiên, 1 số đặc trưng của hệ phân tán thực thụ chưa được thực hiện:

Hạn chế	Lý do trong phạm vi đề tài	Hướng khắc phục production
Chưa có Two-Phase Commit (2PC) hoàn chỉnh	Coordinator đóng vai trò 2PC đơn giản qua reserve/commit/release thủ công	Saga Pattern hoặc distributed transaction manager
Chưa có replication engine thời gian thực	Catalog được seed một lần, giả định không thay đổi trong thời gian demo	PostgreSQL logical replication, Debezium CDC
Coordinator là điểm đơn	Chấp nhận được trong	Nhiều instance FastAPI

lỗi	môi trường demo cục bộ	+ load balancer
Chưa có xử lý network partition	Ba site chạy trên cùng máy, không có nguy cơ mạng thật	Chiến lược CAP theorem theo yêu cầu nghiệp vụ

X. Kết luận và hướng phát triển

10.1. Kết luận

Nhóm đã xây dựng và vận hành thành công hệ thống demo cơ sở dữ liệu phân tán cho bài toán thương mại điện tử đa kho. Hệ thống không chỉ dừng ở lý thuyết mà đã thể hiện trực tiếp các đặc trưng phân tán qua dữ liệu và xử lý thực tế có thể kiểm chứng

Về chiến lược tổ chức dữ liệu, hệ thống phân chia rõ ràng giữa dữ liệu nhân bản và dữ liệu phân mảnh. Dữ liệu catalog ít thay đổi được nhân bản toàn phần để mỗi site tự phục vụ mà không cần truy vấn xuyên site. Dữ liệu giao dịch có tần suất cập nhật cao được phân mảnh ngang theo vùng địa lý, đảm bảo mỗi site chỉ chịu trách nhiệm với phần dữ liệu mình sở hữu, tránh xung đột và chi phí đồng bộ không cần thiết. Chiến lược này thỏa mãn ba tính chất của phân mảnh hợp lệ: đầy đủ, tái tạo được và không giao nhau

Về tầng điều phối, FastAPI coordinator hiện thực mô hình fan-out: gửi truy vấn đồng thời tới nhiều site và tổng hợp thành một response thống nhất, đảm bảo người dùng không nhận thấy sự phân tán bên dưới. Thuật toán phân bổ đơn hàng ưu tiên kho gần vùng khách hàng nhằm tối ưu locality of reference, đồng thời tự động gom hàng từ các site khác khi site ưu tiên không đủ tồn kho. Chu trình reserve/commit/release kết hợp khóa mức dòng SELECT FOR UPDATE đảm bảo ràng buộc available_qty không bao giờ âm kho, ngay cả khi nhiều request tranh chấp cùng một SKU tại cùng thời điểm - điều này đã được xác nhận qua kịch bản demo đồng thời thực tế

Hệ thống đáp ứng đầy đủ yêu cầu đề bài: minh họa thành công phân mảnh ngang, nhân bản dữ liệu, truy vấn phân tán fan-out, điều phối giao dịch đa site và nhất quán trong môi trường đồng thời - tất cả trong một ứng dụng hoàn chỉnh chạy được thực tế và có thể kiểm thử trực tiếp

10.2. Hướng phát triển

10.2.1. Nâng cấp cơ chế giao dịch phân tán

Bước tiếp theo quan trọng nhất là thay thế cơ chế reserve/commit/release thủ công bằng Two-Phase Commit (2PC) hoặc Saga Pattern để quản lý giao dịch đa site một cách chính thức và chịu lỗi tốt hơn. Tích hợp PostgreSQL logical replication hoặc Change Data Capture (CDC) để tự động đồng bộ dữ liệu catalog khi có thay đổi, thay vì phải seed lại thủ công.

10.2.2. Cải thiện độ tin cậy

Bổ sung cơ chế retry với exponential backoff và circuit breaker pattern để coordinator xử lý được trường hợp một site tạm thời không khả dụng mà không ảnh hưởng toàn bộ hệ thống. Triển khai nhiều instance coordinator phía sau load balancer để loại bỏ điểm đơn lỗi.

10.2.3. Mở rộng quy mô và tính năng

Thiết kế lại siteregistry để hỗ trợ thêm site mới qua cấu hình mà không cần thay đổi code. Thêm tầng cache Redis cho dữ liệu catalog nhân bản để giảm độ trễ đọc. Về nghiệp vụ, có thể bổ sung luồng trả hàng và hoàn kho đúng site đã xuất, hệ thống tự động tạo yêu cầu bổ sung hàng khi tồn kho dưới ngưỡng reorder, và dashboard theo dõi hiệu năng truy vấn phân tán theo thời gian thực.